ResearchPaperWorkflow v2

科研论文多智能体协作工作流
完整设计文档

4层架构 · 12类Agent · 18阶段管线 · 16质量门 · 5工作流模式

Version 2.0.0 — June 20, 2026

本文档由 9 份设计文档合并生成，涵盖系统架构、Agent协作、管线调度、质量门、工作流模式、技能集成与案例研究等完整内容。适用于科研论文写作工作流的设计评审、技术选型与团队培训。

目录

README — 工作流概述与快速开始

A general-purpose, agent-driven research paper workflow system for bioinformatics and clinical research — upgraded to evidence-centric medical evidence generation.

What It Does

This framework transforms the research paper writing process from ad-hoc scripting and manual coordination into a **deterministic, auditable, multi-agent pipeline** with medical evidence quality assurance:

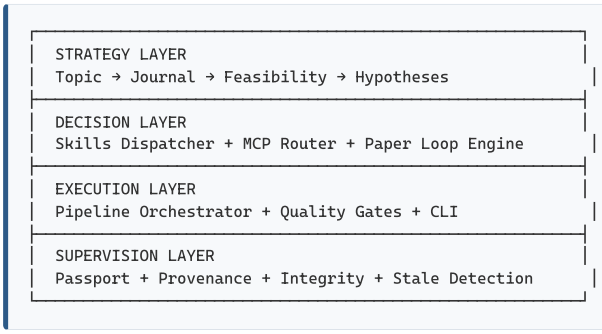
- **4-layer architecture**: Strategy → Decision → Execution → Supervision
- **19-stage paper pipeline**: Including new Statistical Analysis Plan (SAP) pre-specification stage
- **7-framework skills + 28 external skills**: Covering every research phase via the skill registry
- **18 domain-specific agents**: With clear responsibility boundaries (6 new medical agents in v3.0)
- **1 collaborative team** (`paper_writing_team`): For full-cycle manuscript production
- **Passport system**: 5-ledger hash-based artifact tracking, claim binding, and stale detection
- **41 integrity gates**: Automated quality enforcement (17 CRITICAL, 21 HIGH, 3 MEDIUM)
  - Including clinical design, data bias, statistics/model, single-cell/spatial omics, and AI/ML categories
- **Evidence Graph**: Full claim→statistics→artifact→code→parameter traceability
- **CLI surface**: 10 commands for full pipeline control

Design Philosophy

> **Main thread plans & integrates. Subagents explore, review, and execute in parallel. Skills provide reusable workflows. The loop engine manages state and iteration.**

This is NOT a "smarter chatbot"—it's a **project execution system** that reads files, runs code, spawns parallel subagents, and enforces scientific quality gates.

Architecture



Quick Start

```
# Clone
git clone https://github.com/your-org/ResearchPaperWorkflow.git
cd ResearchPaperWorkflow

# Install
pip install -e .

# Create your first paper project
python -m paper_workflow.cli create-project \
  --idea "Your research idea" \
  --field "your field, keywords" \
  --journal "Genome Biology"
```

```
# Check status
python -m paper_workflow.cli status --paper <paper_id>

# Run pipeline
python -m paper_workflow.cli run-pipeline --paper <paper_id>

# Run tests
python tests/test_all.py
```

Usage Patterns

Pattern 1: New Paper from Scratch

```
create-project → search-literature → research-plan → data-audit
→ figure-planning → run-analysis → verify-methods
→ write-methods → write-results → write-introduction → write-discussion
→ assemble-manuscript → integrity-check → internal-review
→ apply-revision → re-review → quality-check → finalize
```

Pattern 2: From Existing Results

```
create-project → record-artifacts → figure-planning
→ write-results → ... → finalize
```

Pattern 3: Revision Cycle

```
diagnose-gate-failures → generate-revision-plan
→ apply-revision → re-review → quality-check
```

Key Features

Loop Engine

- **observe → decide → run → verify → record → mark_stale → diagnose → repeat**
- 18 pipeline stages with dependency tracking
- Automatic stale detection when upstream artifacts change
- Human checkpoints at critical decision points

Passport System

- `project_passport.yaml` — Project identity and metadata
- `artifact_ledger.jsonl` — Append-only artifact hash log
- `checkpoint_ledger.jsonl` — User-approved checkpoints
- `integrity_ledger.jsonl` — Integrity gate events

Integrity Gates (16 rules)

Severity	Rules
CRITICAL	BibTeX existence, citation traceability, Results no-citations, claim-artifact binding, figure references
HIGH	Data availability, code availability, no local paths, parameters complete, limitations discussed, no overinterpretation, statistics reported, pseudoreplication check
MEDIUM	Section length minimum, no bullets in prose, figure count, journal format

Agent System (10 agents)

`research_strategist` · `literature_reviewer` · `data_auditor` · `figure_planner` · `analysis_executor` · `pipeline_engineer` · `statistician` · `report_writer` · `integrity_checker` · `team_orchestrator`

Framework Skills (5 skills)

`topic_research` · `paper_loop` · `figure_planning` · `paper_writing` · `revision_routing`

External Skills (28 skills, see `.claude/SKILL_REGISTRY.md`)

Covering: research · writing · polishing · figures · citations · review · data compliance · detection · presentation · orchestration · domain analysis · meta

Project Structure

```
ResearchPaperWorkflow/
├── src/paper_workflow/          # Core Python package
│   ├── strategy/                # Strategy layer (topic, journal, feasibility, hypothesis)
│   ├── engine/                  # Paper loop engine
│   ├── supervision/             # Passport, integrity gates
│   ├── cli/                     # CLI surface
│   └── workflow.py              # Unified workflow orchestrator
├── .claude/                     # Agent/skill/team definitions
│   ├── SKILL_REGISTRY.md        # 28-skill mapping with trigger words
│   ├── skills/ (5) + agents/ (10) + teams/ (1)
├── config/                      # Configuration files
│   ├── default_config.yaml      # Pipeline configuration
│   ├── journal_database.yaml    # Journal profiles
│   └── templates/               # Paper section templates
├── code_library/                # Reusable analysis code
│   ├── modules/                 # Full analysis modules
│   ├── patterns/                # QC, clustering patterns
│   ├── snippets/                # I/O, logging, config utilities
│   └── solutions/               # Common problem solutions
├── docs/                        # Documentation
├── tests/                       # Integration tests
├── examples/                    # Example project
└── papers/                      # Generated paper projects (gitignored)
```

Supported Paper Types

Type	Description	Typical Pipeline
`original_research`	Primary research article	Full 18-stage pipeline
`methods`	Methods/tool paper	Emphasizes verification + benchmarking
`review`	Literature review	Emphasizes literature search + synthesis
`clinical_research`	Clinical/translational study	Adds ethics + clinical statistics gates
`data_resource`	Data/resource descriptor	Emphasizes data audit + availability
`brief_communication`	Short report	Condensed pipeline, stricter limits

Supported Research Domains

The framework is domain-agnostic but pre-configured for:

- **Bioinformatics & Computational Biology**
- **Spatial Transcriptomics & Single-Cell Genomics**
- **Clinical & Translational Research**
- **Multi-Omics Integration**
- **Machine Learning in Biomedicine**

Configuration is fully externalized — adapt to any research domain by modifying `config/default_config.yaml` and `config/journal_database.yaml`.

Integration with AI Assistants

This framework is designed to work with AI coding assistants (Claude Code, Codex, etc.):

1. **AGENTS.md**: Assistant reads this on startup — defines project rules
2. **Skills**: Assistant invokes skills for specialized tasks
3. **MCP Tools**: Connects to PubMed, Consensus, Context7, etc.
4. **Subagents**: Assistant spawns specialized subagents for parallel work

License

Citation

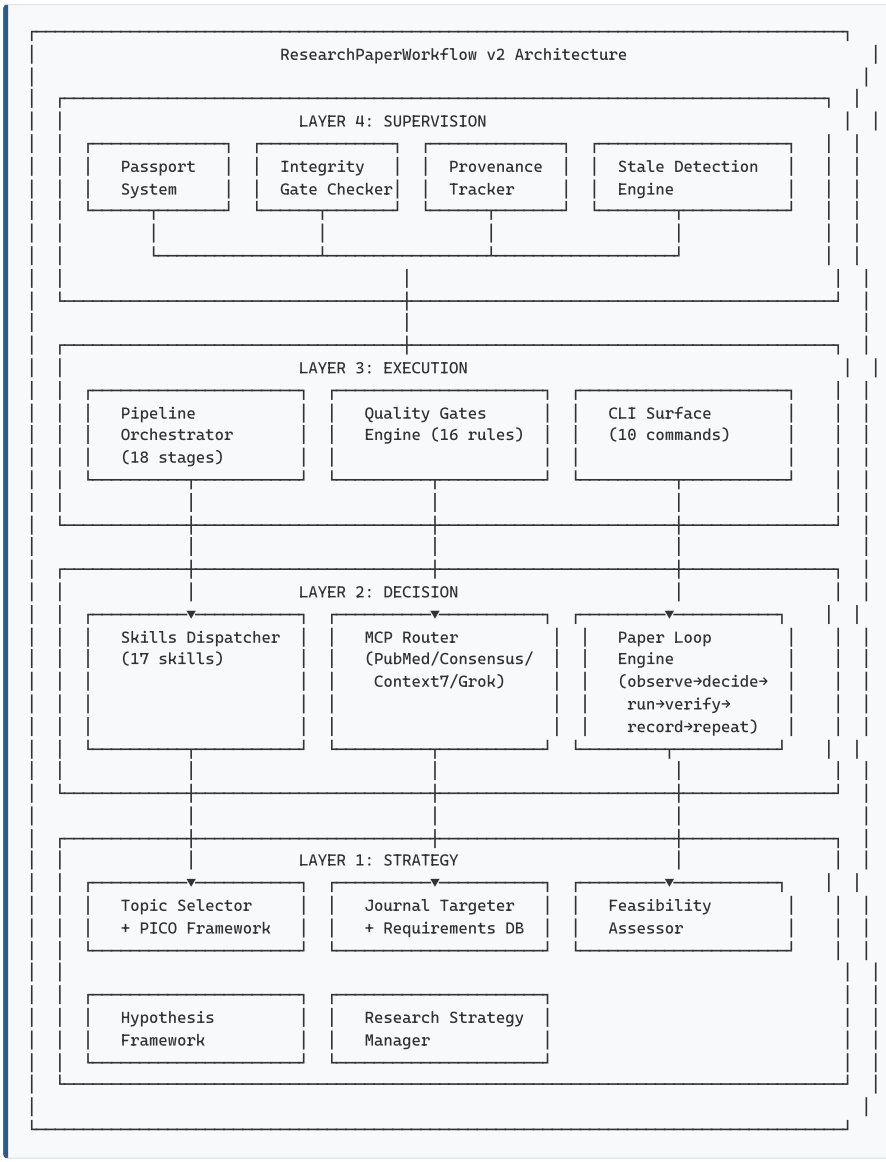
If you use this framework in your research, please cite:

```
@software{research_paper_workflow_2026,
  title = {Research Paper Workflow Framework},
  year = {2026},
  url = {https://github.com/your-org/ResearchPaperWorkflow}
}
```

Built on the loop-engineering principles of Draftpaper_loop and the three-layer architecture pattern.

ARCHITECTURE — 4层系统架构

1. 架构全景



2. 四层架构详解

2.1 Layer 1: Strategy（策略层）

职责: 研究问题定义、期刊选择、可行性评估、假设生成

组件	源码位置	功能
`TopicSelector`	`src/paper_workflow/strategy/topic_selector.py`	从研究想法生成结构化主题，评估创新度
`JournalTargeter`	`src/paper_workflow/strategy/journal_targeter.py`	匹配目标期刊，生成格式要求清单
`FeasibilityAssessor`	`src/paper_workflow/strategy/feasibility.py`	多维度可行性评分（数据/方法/期刊/时间线）
`HypothesisFramework`	`src/paper_workflow/strategy/hypothesis_framework.py`	结构化假设生成（category + type + statement）
`ResearchStrategyManager`	`src/paper_workflow/strategy/research_strategy.py`	策略持久化、加载、版本管理

数据流:

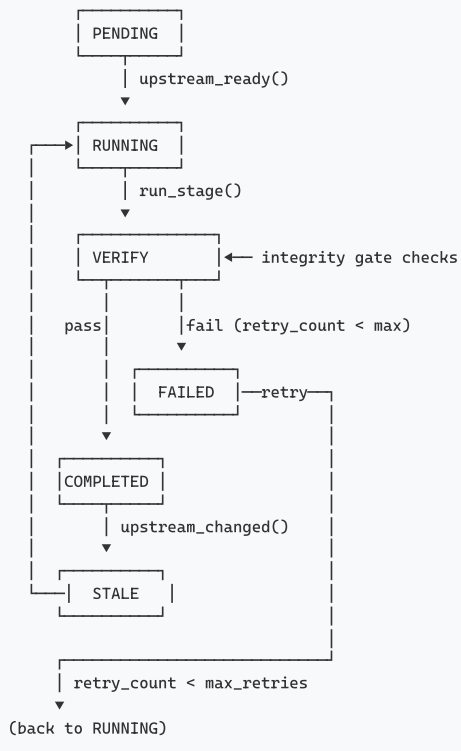
```
idea + field → TopicSelector → Topic (innovation_level, scope, knowledge_gaps)
                                ↓
                            JournalTargeter → Journal (IF, fit_score, requirements)
                                ↓
                            HypothesisFramework → [Hypothesis] (PICO-structured)
                                ↓
                            FeasibilityAssessor → FeasibilityReport (go/no-go)
```

2.2 Layer 2: Decision（决策层）

职责: 技能调度、MCP路由、管线循环控制

组件	源码位置	功能
`SkillsDispatcher`	`config/default_config.yaml §6`	触发词→技能映射，17技能覆盖6阶段
`MCP Router`	内置（Claude Code harness）	PubMed/Consensus/Context7/Grok 路由
`PaperLoopEngine`	`src/paper_workflow/engine/loop_engine.py`	observe→decide→run→verify→record→sync 循环
`AgentDispatcher`	`src/paper_workflow/engine/agent_dispatcher.py`	Stage→Agent→Skill 三级调度

Loop Engine 状态机:



Pipeline State 转换:

```
CLEAN → IN_PROGRESS → CLEAN (all stages OK)
CLEAN → IN_PROGRESS → GATE_FAILURE (integrity check failed)
CLEAN → IN_PROGRESS → STALE_STAGES (upstream artifact changed)
CLEAN → IN_PROGRESS → BLOCKED (unresolvable dependency)
CLEAN → IN_PROGRESS → DRIFT_DETECTED (artifact hash mismatch)
```

2.3 Layer 3: Execution（执行层）

职责: 18阶段管线编排、质量门执行、CLI命令接口

组件	源码位置	功能
`PaperWorkflow`	`src/paper_workflow/workflow.py`	统一工作流编排器（初始化→运行→诊断）
`E2EWorkflow`	`src/paper_workflow/e2e_workflow.py`	5阶段端到端主控（含回调+检查点+报告）
`IntegrityGateChecker`	`src/paper_workflow/supervision/integrity.py`	16规则完整性检查（5C+8H+3M）
CLI	`src/paper_workflow/cli/main.py`	10命令CLI界面

E2E 5-Phase 结构:

```
Phase 1: Topic Research (4 stages)
  deep_research → topic_research → journal_targeting → feasibility_assessment
  [CHECKPOINT: Review research question and approve]

Phase 2: Data Analysis (7 stages)
  data_audit → qc_filtering → clustering → cell_annotation
  → statistical_testing → pathway_inference → figure_planning
  [CHECKPOINT: Review analysis outputs and approve]

Phase 3: Paper Writing (4 stages)
  scientific_writing_plan → nature_writing → nature_citation → research_paper_writing
  [CHECKPOINT: Review complete manuscript draft]

Phase 4: Polish & Review (6 stages)
  academic_polish → humanizer → nature_figure → academic_review
  → nature_data → ai_writing_detection
  [CHECKPOINT: Review polished manuscript + review feedback]

Phase 5: Submission (3 stages)
  cover_letter → integrity_check → presentation (optional)
  [CHECKPOINT: Final review before submission]
```

2.4 Layer 4: Supervision（监督层）

职责: 护照追踪、完整性验证、溯源审计、过期检测

组件	源码位置	功能
`PaperPassport`	`src/paper_workflow/supervision/passport.py`	Hash-based artifact ledger + checkpoint记录
`IntegrityGateChecker`	`src/paper_workflow/supervision/integrity.py`	16-rule 完整检查套件
`ErrorTracker`	`src/paper_workflow/utils/error_tracker.py`	结构化错误日志 + 分级
`Reproducibility`	`src/paper_workflow/utils/reproducibility.py`	种子验证 + 环境快照

Passport 数据结构:

```
papers/<paper_id>/
├── project_passport.yaml        # 项目身份 + 元数据
├── artifact_ledger.jsonl        # Append-only 制品哈希日志
├── checkpoint_ledger.jsonl      # 用户批准的检查点
└── integrity_ledger.jsonl       # 完整性门事件
```

---

3. 核心数据流

3.1 论文生产全流程数据流

```
Research Idea
     │
     ▼
[TopicSelector] ──→ Topic (innovation_level, scope, gaps)
     │
     ▼
[JournalTargeter] ──→ Journal (IF, fit_score, formatting reqs)
     │
     ▼
[Literature Searcher] ──→ references.bib + citation_evidence.csv
     │
     ▼
[HypothesisFramework] ──→ hypotheses.yaml + research_questions
     │
     ▼
[Data Auditor] ──→ data_audit_report.md + data_inventory.yaml
```

```
     |
     ▼
[Figure Planner] ──→ figure_plan.json + figure_specs.yaml
     |
     ▼
[Analysis Executor] ──→ results/* + run_manifest.yaml
     |
     ▼
[Method Verifier] ──→ methods/run_manifest.yaml (verified)
     |
     ▼
[Report Writer] ×4 ──→ manuscript/{intro,methods,results,discussion}.md
     |
     ▼
[Assembler] ──→ manuscript/manuscript.tex + manuscript.pdf
     |
     ▼
[Integrity Checker] ──→ integrity/integrity_report.json
     |
     ▼
[Internal Reviewer] ──→ review/review_report.md
     |
     ▼
[Revision Applier] ──→ manuscript/manuscript_revised.tex
     |
     ▼
[Re-Reviewer] ──→ review/re_review_report.md
     |
     ▼
[Finalizer] ──→ submission/manuscript_final.pdf + cover_letter.pdf
```

3.2 Agent 间通信数据流

```
research_strategist ──(topic, hypotheses)──→ literature_reviewer
        |                                          |
        |                                   (citation_evidence)
        |                                          |
        ▼                                          ▼
  data_auditor ←──────────────→ figure_planner
        |                           |
   (data_inventory)           (figure_plan)
        |                           |
        ▼                           ▼
  analysis_executor ──(results)──→ pipeline_engineer
        |                              |
        |                        (verified_manifest)
        |                              |
        ▼                              ▼
  report_writer ←──────────────────────┘
        |
   (manuscript sections)
        |
        ▼
  integrity_checker ──(integrity_report)──→ team_orchestrator
        |                                        |
        |                                  (review_report)
        |                                        |
        ▼                                        ▼
  report_writer (revision) ←─────────────────────┘
        |
   (revised_manuscript)
        |
        ▼
  integrity_checker (final) ──→ submission_package
```

---

4. 状态管理

4.1 StageState（管线阶段状态）

```python
@dataclass
class StageState:
    definition: StageDefinition    # 阶段定义（不可变）
    status: StageStatus            # PENDING|RUNNING|COMPLETED|FAILED|STALE|SKIPPED|BLOCKED
    started_at: Optional[str]
    completed_at: Optional[str]
    retry_count: int
    artifacts_produced: list[str]  # 产出文件路径
    artifact_hashes: dict[str, str]  # SHA-256 哈希
```

```
    errors: list[str]
    gate_results: list[dict]       # 质量门检查结果
```

4.2 WorkflowState（工作流状态）

```
@dataclass
class WorkflowState:
    paper_id: str
    started_at: str
    completed_at: Optional[str]
    strategy: Optional[ResearchStrategy]
    pipeline_state: str            # clean|drift_detected|stale_stages|gate_failure|in_progress|blocked
    stages_completed: int
    stages_failed: int
    errors: list[dict]
    decisions: list[dict]          # 每个阶段的决策记录
```

4.3 状态持久化

```
每次 run() 调用后:
  ┌─ _execute_stage(stage) → self.engine.run_stage()
  │   └─ 写入 project_passport.yaml (updated_at + stages snapshot)
  ├─ verify_stage(stage) → self.engine.verify_stage()
  │   └─ integrity gate checks
  ├─ passport.record_artifact(art, stage=stage)
  │   └─ Append to artifact_ledger.jsonl (SHA-256 hash + timestamp)
  ├─ engine.record_and_sync()
  │   └─ _update_passport() + _sync_stale()
  └─ state.decisions.append( ... )
      └─ 内存中累积，save_state() 时写入 workflow_state/*.json
```

---

5. 可扩展性设计

5.1 扩展点（来自 `default_config.yaml §12`）

扩展类型	注册方式	约束
自定义分析脚本	放入 `scripts/custom/` 自动发现	语法检查 + 导入验证
自定义 Agent	`config/custom_agents.yaml` 清单	必须声明 primary_skills + stages
自定义 Skill	`config/custom_skills.yaml` 清单	必须声明 triggers + phase + agent
自定义 Gate Rule	在 quality_gates 块添加	声明 severity + check type + rule

5.2 配置覆盖链

```
default_config.yaml (基線)
    ↓
<paper_dir>/paper_config.yaml (项目级覆盖)
    ↓
环境变量 PAPER_WORKFLOW_* (运行时覆盖)
    ↓
CLI --config 参数 (命令行覆盖)
```

---

6. 技术栈

层面	技术选择
语言	Python 3.9+ (核心管线) + R 4.0+ (分析模块)
配置	YAML (default_config.yaml, 2100+ 行)
状态持久化	JSONL (append-only ledgers) + YAML (passport)
制品哈希	SHA-256
包管理	pip (Python) + renv (R)
容器化	Docker (Dockerfile.template)
测试	pytest (test_all.py + test_integration.py)
CLI	argparse (10 命令)
输出格式	Markdown, LaTeX, PDF, SVG, JSON, YAML

---

7. 关键设计决策

决策 1: 硬编码 + 配置双轨制

- **选择**: `loop_engine.py` 内置 `PIPELINE_STAGES` 硬编码回退 + YAML 配置覆盖
- **理由**: 配置损坏时管线仍可运行；配置可用时灵活调整阶段定义
- **实现**: `PaperLoopEngine.__init__` 的 `config_path` 参数

决策 2: Append-Only Ledger

- **选择**: `artifact_ledger.jsonl` 和 `checkpoint_ledger.jsonl` 使用 append-only 模式
- **理由**: 完整审计追踪；支持时间旅行调试；不可篡改
- **代价**: 文件持续增长（通过 checkpoint 归档管理）

决策 3: 阶段级 Gate 而非管线级 Gate

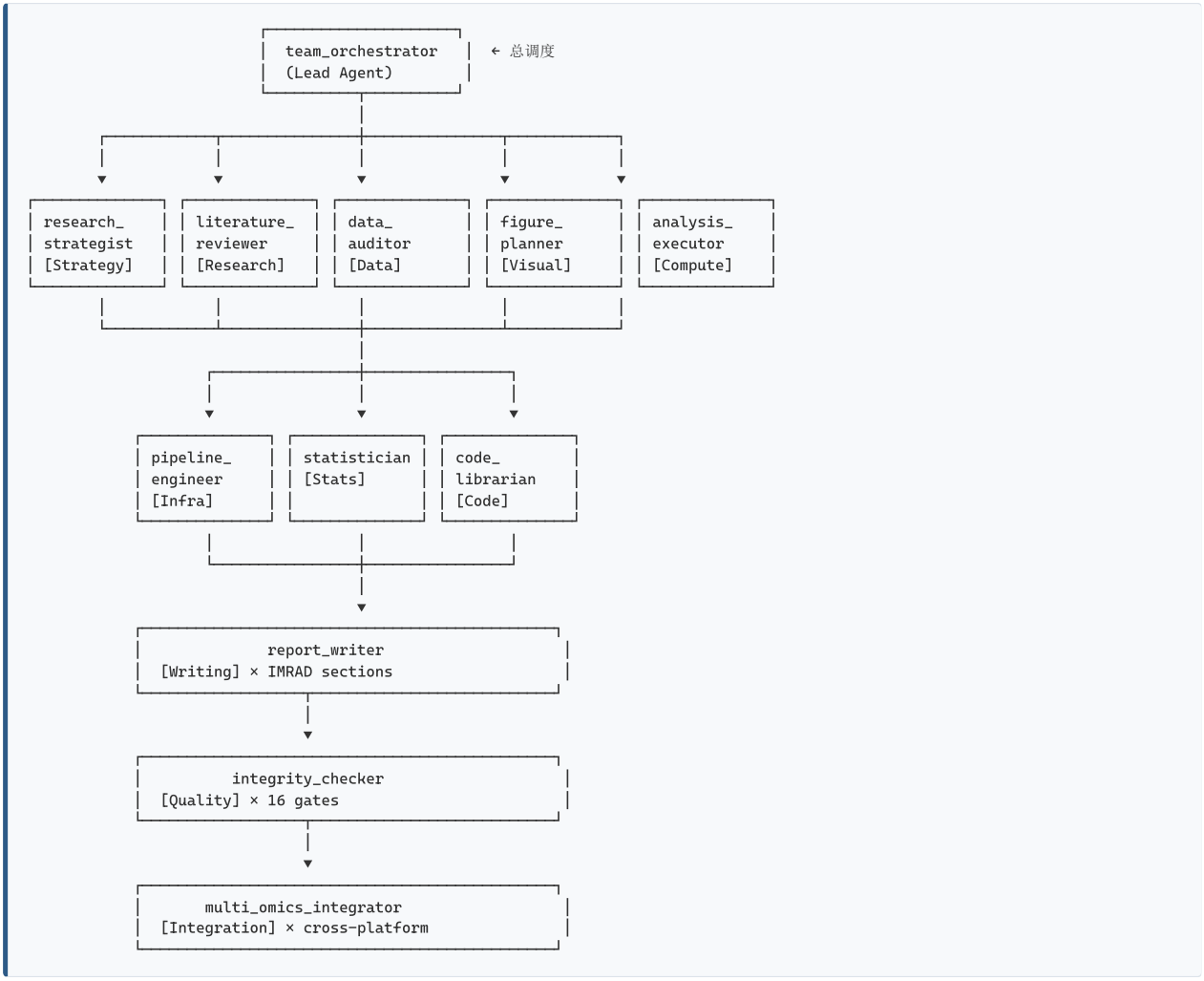
- **选择**: 每个 StageDefinition 携带自己的 gate_rules
- **理由**: 不同阶段有不同质量要求；失败隔离更精确
- **实现**: `StageDefinition.gate_rules: list[dict]`

决策 4: Human Checkpoint 而非全自动

- **选择**: 关键阶段（select_topic, formulate_hypotheses, figure_planning, internal_review, finalize）标记 `human_checkpoint: true`
- **理由**: 科研决策不能全自动化；人在回路保证质量
- **实现**: `stop_at_checkpoint=True` 时在 checkpoint 阶段暂停

AGENT_ROLES — 12类Agent协作体系

1. Agent 体系总览



2. Agent 详细定义

2.1 research_strategist（研究策略师）

属性	值
定位	Strategy Layer 核心，定义研究方向
负责阶段	select_topic, target_journal, formulate_hypotheses
主要技能	topic_research, deep-research, nature-academic-search
输入	研究想法 (idea), 领域 (field)
输出	Topic对象, Journal推荐, Hypothesis列表, Feasibility报告
协作对象	→ literature_reviewer（传递研究问题和知识空白）
人机交互	select_topic + formulate_hypotheses 需要人工 checkpoint

决策能力:

- 评估创新度 (innovation_level: 1-5)

- 推荐匹配期刊 (fit_score: 1-5)
- Go/No-Go 可行性判断
- PICO 框架假设结构化

配置片段 (来自 `default_config.yaml`):

```
research_strategist:
  description: "Designs research strategy, formulates hypotheses, selects target journals"
  primary_skills: [topic_research, deep-research, nature-academic-search]
  stages: [select_topic, target_journal, formulate_hypotheses]
```

2.2 literature_reviewer（文献审阅者）

属性	值
定位	Research Layer 核心，系统性文献检索与综合
负责阶段	literature_search
主要技能	deep-research, nature-academic-search, nature-citation
输入	研究主题、知识空白列表
输出	references.bib, citation_evidence.csv, literature_review.md
协作对象	research_strategist → （接收假设）→ report_writer（传递引用证据）
超时设置	3600秒（文献检索耗时最长）

关键工作流:

1. 多源检索 (PubMed + Semantic Scholar + Scopus + arXiv)
2. 去重与质量管理
3. 引文证据映射 (claim → citation evidence)
4. BibTeX 库生成与验证

2.3 data_auditor（数据审计师）

属性	值
定位	Data Layer，数据质量守门人
负责阶段	data_audit
主要技能	nature-data, qc_pipeline
输入	数据路径、格式声明
输出	data_audit_report.md, data_inventory.yaml, data_availability_statement.md
协作对象	analysis_executor → figure_planner →

审计维度:

- 完整性: 所有声明文件存在且可读
- 格式: 符合预期 schema (h5ad, CSV, FASTQ...)
- 质量: 基本 QC 指标 (MT%, 基因数, UMI数)
- 合规: 人类/动物数据伦理声明
- 可复现: 原始数据来源记录 (GEO/SRA accession)

2.4 figure_planner（图表规划师）

属性	值
定位	Visual Layer，论文叙事视觉化
负责阶段	figure_planning
主要技能	figure_planning, nature-figure
输入	假设列表、分析结果预览
输出	figure_plan.json, figure_plan.html, figure_specs.yaml
协作对象	data_auditor → （接收数据概况）→ analysis_executor（传递图表需求）
人机交互	checkpoint: 审核图表规划

Figure Plan 结构 (6-8 图标准):

```
{
  "Figure 1": {"title": "Study overview and data landscape", "panels": ["A-D"], "type": "schematic + UMAP"},
  "Figure 2": {"title": "Differential expression analysis", "panels": ["A-F"], "type": "volcano + heatmap"},
  "Figure 3": {"title": "WGCNA module discovery", "panels": ["A-E"], "type": "dendrogram + heatmap"},
  "Figure 4": {"title": "Module-trait associations", "panels": ["A-D"], "type": "correlation matrix"},
  "Figure 5": {"title": "Hub gene identification", "panels": ["A-C"], "type": "network + bar"},
  "Figure 6": {"title": "Validation and clinical relevance", "panels": ["A-D"], "type": "ROC + boxplot"}
}
```

2.5 analysis_executor（分析执行器）

属性	值
定位	Compute Layer，核心计算引擎
负责阶段	run_analysis
主要技能	spatial_analysis, statistical_testing, pathway_inference, qc_pipeline
输入	已审计数据 + 图表规划
输出	analysis_results.yaml, statistical_outputs/, *figures/*
超时设置	7200秒（计算密集型）
重试次数	3次

计算管线:

```
raw_data.h5ad
  → QC filtering (mt_filter.py)
  → Normalization + HVG selection
  → Leiden clustering (multi_resolution.py)
  → Cell type annotation (cell_type_annotation.py)
  → Differential expression (statistical_testing)
  → Pathway enrichment (pathway_inference)
  → Results packaging (run_manifest.yaml)
```

2.6 pipeline_engineer（管线工程师）

属性	值
定位	Infra Layer，可复现性保障
负责阶段	run_analysis, verify_methods
主要技能	qc_pipeline, spatial_analysis
输入	分析结果 manifest
输出	methods_verification_report.md
Gate Rules	all_outputs_exist (CRITICAL), code_reproducible (CRITICAL)

2.7 statistician（统计师）

属性	值
定位	Validation Layer，统计方法审核
负责阶段	verify_methods
主要技能	statistical_testing
输入	DE结果、统计报告
输出	统计审核意见

审核清单:

- 检验假设满足（正态性、方差齐性、独立性）
- 多重检验校正正确（Bonferroni/FDR/BH）
- 效应量 + 置信区间完整
- 伪重复检查（生物 vs 技术重复）
- 样本量/功效分析

2.8 code_librarian（代码管理员）

属性	值
定位	Code Layer，代码资产管理
负责阶段	run_analysis, verify_methods, finalize
主要技能	qc_pipeline
输入	代码库路径
输出	插件注册、依赖锁定、代码文档

管理范围:

- `code_library/patterns/` — 可复用模式 (QC, clustering)
- `code_library/modules/` — 完整分析模块
- `code_library/solutions/` — 常见问题解决方案
- `code_library/snippets/` — 工具函数

2.9 report_writer（报告撰写者）

属性	值
定位	Writing Layer，论文文本生产
负责阶段	write_methods, write_results, write_introduction, write_discussion, assemble_manuscript, apply_revision
主要技能	scientific-writing, nature-writing, nature-polishing, humanizer, nature-citation, nature-response
输入	已验证的方法+结果，文献库
输出	IMRAD 各部分 Markdown，完整手稿 LaTeX/PDF
Gate Rules	no_local_paths, parameters_complete, citations_exist, limitations_discussed

写作管线:

```
Methods (from verified pipeline)
   ↓ 参数提取 + 代码引用
Results (from analysis outputs)
   ↓ 数值追溯 + 图表交叉引用 + 禁止解释
Introduction (from literature + hypotheses)
   ↓ 漏斗结构: 背景→空白→假设→目标
Discussion (from results + literature comparison)
   ↓ 总结→对比→局限→意义→展望
Assembly → manuscript.tex + manuscript.pdf
```

2.10 integrity_checker（完整性检查者）

属性	值
定位	Quality Layer，最终质量守门人
负责阶段	verify_methods, integrity_check, internal_review, re_review
主要技能	qc_pipeline, academic-paper-reviewer, nature-data, nature-citation
Gate 执行	16 规则 × 3 严重级别

Gate 执行流程:

```
1. 加载手稿 sections (intro, methods, results, discussion)
2. 加载 BibTeX 库 (references/library.bib)
3. 加载图表计划 (figure_plan.json)
4. 加载期刊目标 (formatting_requirements.yaml)
5. 逐规则执行检查
6. 生成 IntegrityReport
   ├── passed: bool
   ├── critical_failures: int  → blocks_pipeline = True
   ├── high_failures: int      → blocking but fixable
   ├── medium_failures: int    → warning only
   └── results: [{rule, passed, message}]
```

2.11 multi_omics_integrator（多组学整合师）

属性	值
定位	Integration Layer，跨平台数据整合
负责阶段	run_analysis, verify_methods
主要技能	multi_omics, spatial_analysis, pathway_inference
输入	多组学数据集
输出	跨平台整合结果

整合方法库:

- 跨平台归一化 (ComBat, Harmony, scVI)
- 联合降维 (MOFA, Multi-Omics Factor Analysis)
- 多视图聚类 (Spectrum, SNF)
- 生物网络推断 (multi-omics network)

2.12 team_orchestrator（团队调度者）

属性	值
定位	Lead Agent，全局协调
负责阶段	finalize（总结）, 作为 fallback agent
主要技能	deep-research（全局视角）
输入	完整管线状态
输出	最终提交包
特殊能力	跨 Agent 协调、冲突解决、回退处理

2.13 clinical_methodologist（临床方法学家）— v3.0 NEW

属性	值
定位	研究设计审核，PICO/PECO/PIRD框架，偏倚评估，报告指南选择
负责阶段	design_analysis_plan, data_audit
主要技能	研究设计分类、偏倚风险评估、终点定义验证、报告指南匹配
输入	hypotheses.yaml, research_questions.yaml, clinical_value_matrix.yaml
输出	study_design_protocol.yaml, design_assessment_report.md

2.14 ethics_compliance_auditor（伦理合规审计师）— v3.0 NEW

属性	值
定位	IRB/伦理批准验证，数据隐私合规，ICMJE AI声明，利益冲突审核
负责阶段	data_audit, integrity_check, finalize
主要技能	伦理批准文件验证、知情同意审核、数据隐私合规、临床试验注册验证
输入	study_design_protocol.yaml, ethics_documentation/
输出	ethics_compliance_report.md, compliance_checklist.yaml

2.15 causal_inference_reviewer（因果推断审查员）— v3.0 NEW

属性	值
定位	DAG审查，混杂评估，工具变量假设验证，敏感性分析(E-value)，因果语言审核
负责阶段	design_analysis_plan, verify_methods, write_discussion
主要技能	DAG后门路径识别、IV三假设验证、E-value计算、中介分析假设检查
输入	statistical_analysis_plan.yaml, hypotheses.yaml, analysis_results
输出	causal_assumption_audit.md, causal_evidence_rating.yaml

2.16 external_validation_planner（外部验证规划师）— v3.0 NEW

属性	值
定位	独立队列验证策略设计，样本独立性验证，跨中心/跨平台验证规划
负责阶段	design_analysis_plan, verify_methods, integrity_check
主要技能	验证层级设计(内部→时间→地理→外部)、验证队列识别、性能阈值设定
输入	study_design_protocol.yaml, statistical_analysis_plan.yaml
输出	validation_plan.yaml, validation_cohort_candidates.md

2.17 reviewer_simulator（多角色审稿模拟器）— v3.0 NEW

属性	值
定位	4角色独立审稿模拟(EIC+统计+临床+生信)，致命缺陷检测，反辩策略生成
负责阶段	internal_review, re_review
主要技能	多角色独立审稿、投稿准备度评分(0-100)、反辩要点预生成
输入	manuscript_draft.md, figures/, journal_requirements.yaml
输出	simulated_peer_review.md, rebuttal_talking_points.md

2.18 novelty_killer（创新杀手/红队）— v3.0 NEW

属性	值
定位	对抗性审查——主动寻找致命缺陷：样本不足、无外部验证、ML无增量、机制链断裂
负责阶段	formulate_hypotheses, internal_review
主要技能	10类致命缺陷系统评估(FATAL/MAJOR/MINOR/ABSENT)、竞争力对比、拒稿风险评分
输入	hypotheses.yaml, data_inventory.yaml, study_design_protocol.yaml
输出	fatal_flaws_report.md, rejection_risk_summary.yaml

3. Agent 协作模式

3.1 串联协作 (Sequential Handoff)

```
research_strategist → literature_reviewer → report_writer → integrity_checker

每个 Agent 完成工作后，下一个 Agent 以上一个的输出为输入。
适用场景：线性无分支的任务（如写 Introduction）。
```

3.2 扇出协作 (Fan-Out Parallel)

```
                        ┌──→ analysis_executor (clustering)
data_auditor ──────────┼──→ analysis_executor (DE)
                        └──→ analysis_executor (pathway)

                    并行执行独立分析，结果汇聚到 figure_planner。
                    适用场景：多分析同时运行。
```

3.3 交叉验证协作 (Adversarial Verify)

```
report_writer ──→ manuscript_draft
                          │
          ┌───────────────┼───────────────┐
          ▼               ▼               ▼
  integrity_checker  statistician   literature_reviewer
  (格式+引用)        (统计完整性)    (文献准确性)
          │               │               │
          └───────────────┼───────────────┘
                          ▼
                  team_orchestrator
                  (冲突裁决 + 综合报告)
```

3.4 迭代修订协作 (Loop-until-Clean)

```
report_writer → integrity_checker → [PASS?]
                                      ├─ YES → team_orchestrator
                                      └─ NO  → report_writer (revise)
                                               ↓
                                          integrity_checker → ...
                          最多迭代 max_retries=5 次 (apply_revision 阶段)
```

---

4. Agent 到 Stage 的映射

Stage (18)	Phase	Primary Agent	Support Agents
select_topic	1	research_strategist	—
target_journal	1	research_strategist	—
literature_search	1	literature_reviewer	—
formulate_hypotheses	1	research_strategist	literature_reviewer
data_audit	2	data_auditor	code_librarian
figure_planning	2	figure_planner	data_auditor
run_analysis	2	analysis_executor	pipeline_engineer, multi_omics_integrator, code_librarian
verify_methods	2	pipeline_engineer	statistician, integrity_checker
write_methods	3	report_writer	pipeline_engineer
write_results	3	report_writer	figure_planner
write_introduction	3	report_writer	literature_reviewer
write_discussion	3	report_writer	literature_reviewer
assemble_manuscript	4	report_writer	—
integrity_check	4	integrity_checker	—
internal_review	4	team_orchestrator	integrity_checker
apply_revision	5	report_writer	—
re_review	5	team_orchestrator	integrity_checker
finalize	6	integrity_checker	team_orchestrator, code_librarian

5. Agent 通信协议

5.1 Artifact-Based Communication

Agent 之间**不直接通信**，而是通过写入/读取共享制品文件交换信息:

```
Agent A 写入: papers/<id>/results/de_results.csv
                ↓ (artifact_ledger.jsonl 记录 hash)
Agent B 读取: papers/<id>/results/de_results.csv
                ↓ (验证 hash 一致性)
Agent B 处理并写入新制品
```

5.2 Passport 同步

```
每次 stage 完成后:
  1. AgentDispatcher.dispatch() → StageResult (含 artifacts + errors)
  2. PaperWorkflow._execute_stage() → passport.record_artifact()
  3. PaperLoopEngine.record_and_sync() → _update_passport() + _sync_stale()
  4. 下游 stage 的 upstream_ready() 检查上游 status == COMPLETED
```

5.3 过期检测

```
当上游 stage 的 artifact hash 变化:
  1. PaperPassport.detect_artifact_drift() 检测到 hash 不匹配
  2. 所有下游 stage 的 status → STALE
  3. PipelineState → STALE_STAGES
  4. decide_next_stage() 优先返回 stale stages (重运行)
```

6. Paper Writing Team（协作团队）

6.1 团队定义

`paper_writing_team` 是一个预配置的协作团队，包含全部 12 个 Agent，用于完整的论文生产周期。

触发: 当用户请求 "写完整论文" / "full paper workflow" / "academic pipeline"

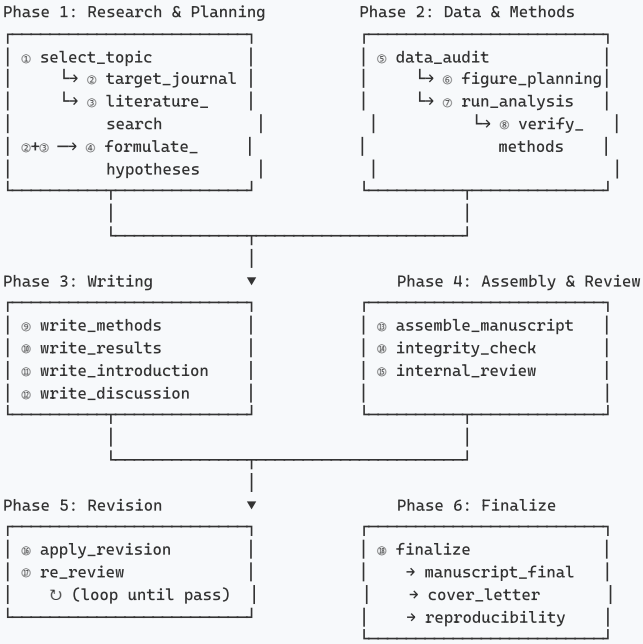
协调模式: team_orchestrator 作为 Lead，分配任务给其他 Agent

6.2 团队工作流

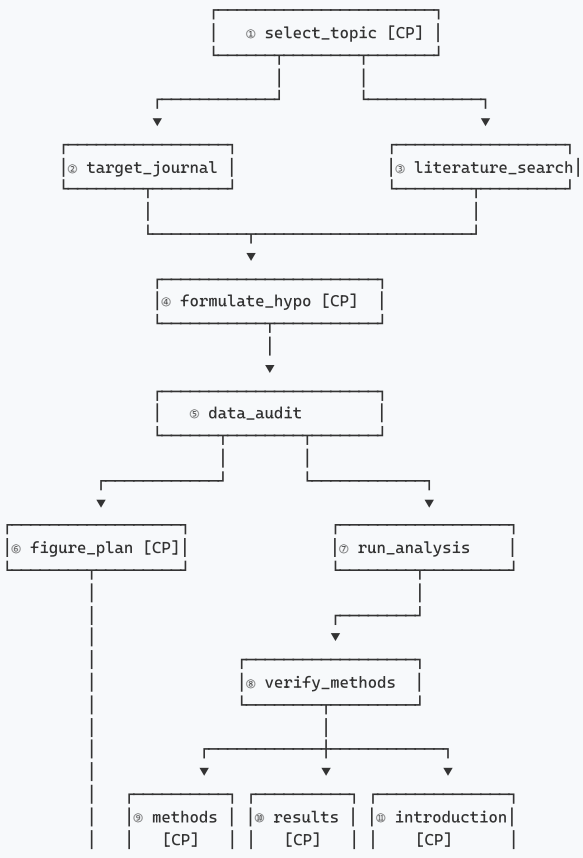
```
Phase 1: Research  ──┬── research_strategist + literature_reviewer (并行)
Phase 2: Data      ──┬── data_auditor → analysis_executor → pipeline_engineer
Phase 3: Writing   ──┬── report_writer (串联 IMRAD)
Phase 4: Review    ──┬── integrity_checker + statistician (交叉验证)
Phase 5: Finalize  ──┬── team_orchestrator + integrity_checker
```

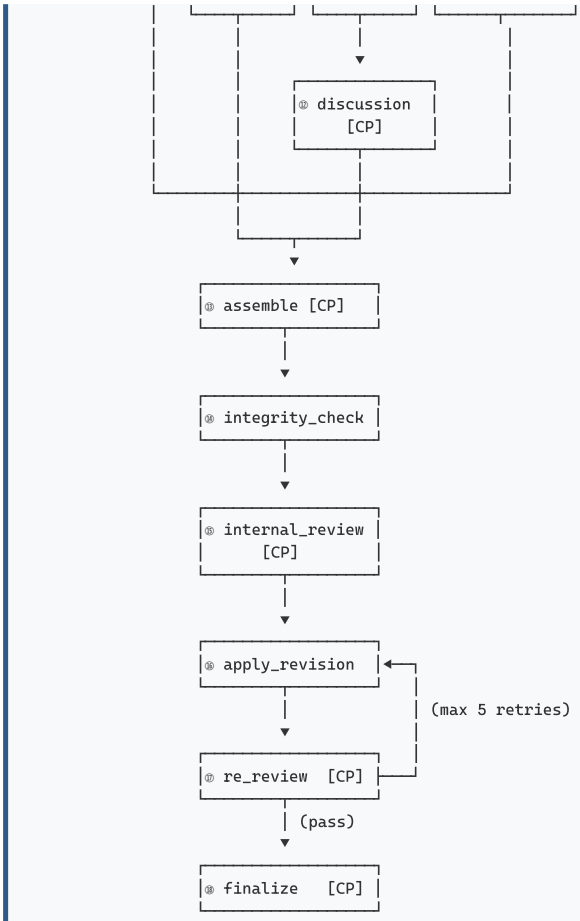
PIPELINE_DESIGN — 18阶段调度管线

1. 管线全景：18-Stage Pipeline



2. 阶段依赖图 (DAG)





[CP] = Human Checkpoint (需要人工审核批准)

---

3. 并行调度策略

3.1 阶段内并行 (Intra-Phase Parallelism)

Phase 3 Writing 的4个写作为什么可以并行:

```
⑧ verify_methods ──┬──→ ⑨ write_methods       (depends: ⑧)
                   └──→ ⑩ write_results       (depends: ⑧ + ⑥)

③ literature ──────┬──→ ⑪ write_introduction  (depends: ③ + ④)
④ hypotheses       │
                   └──→ ⑫ write_discussion    (depends: ⑩ + ③)
                        (⑩ 必须 COMPLETED)

并行窗口:
  t0: ⑧ COMPLETED → ⑨ + ⑩ 可同时启动 (⑨只需⑧, ⑩需要⑧+⑥)
  t1: ⑩ COMPLETED → ⑫ 启动 (⑫ 依赖 ⑩)
  t0-t1 期间: ⑨ + ⑪ 可以并行 (⑨ 在用⑧, ⑪ 在用③+④)
```

3.2 decide_next_stage() 调度逻辑

```
def decide_next_stage(self) -> Optional[str]:
    for stage_def in self._active_stages:          # 按 phase order 扫描
        stage = self.stages[stage_def.name]
        if stage.status == StageStatus.COMPLETED:   # 跳过已完成
            continue
        if stage.status == StageStatus.FAILED and \
           stage.retry_count ≥ stage_def.max_retries:  # 跳过重试耗尽
            continue
        upstream_ready = all(
            self.stages[u].status == StageStatus.COMPLETED
            for u in stage_def.upstream              # 检查所有上游
        )
        if upstream_ready:
            return stage_def.name                    # 返回第一个就绪阶段
```

```
    # 全完成 → CLEAN，否则 BLOCKED
    return None
```

关键特性: 非贪心 — 按 order 顺序选择，不跳级。保证逻辑顺序前提下最大化并行度。

---

4. 工作路径设计（5种模式）

4.1 完整研究路径 (Full Research)

```
Idea → ① Topic → ② Journal → ③ Literature → ④ Hypotheses [CP1]
  → ⑤ Data Audit → ⑥ Figure Plan [CP2] → ⑦ Analysis
  → ⑧ Verify → ⑨ Method → ⑩ Results → ⑪ Intro → ⑫ Discussion [CP3]
  → ⑬ Assemble → ⑭ Integrity → ⑮ Review [CP4]
  → ⑯ Revise → ⑰ Re-review [CP5]
  → ⑱ Finalize [CP6]

适用: 全新项目，有原始数据，从头写论文
耗时: 8-12 周
调用: wf.run(phases=[1,2,3,4,5])
```

4.2 快速写作路径 (From Results)

```
已有结果 → ⑤ Data Audit (简略) → ⑥ Figure Plan [CP]
  → ⑨ Methods → ⑩ Results → ⑪ Intro → ⑫ Discussion
  → ⑬ Assemble → ⑭ Integrity → ⑮ Review [CP]
  → ⑯ Revise → ⑰ Re-review → ⑱ Finalize

适用: 分析代码已完成，结果在手，只需写论文
耗时: 2-4 周
调用: wf.run(phases=[3,4,5])
```

4.3 修订循环路径 (Revision Cycle)

```
Reviewer Comments
  → ⑯ Apply Revision → ⑰ Re-review
  ↻ loop: 如果 re-review 不通过，修改后重来 (最多5次)
  → ⑱ Finalize

适用: 收到审稿意见需要修回
耗时: 1-2 周
调用: wf.run(phases=[5])
```

4.4 方法学论文路径 (Methods Paper)

```
跳过 hypothesis + figure_planning + intro writing
保留: analysis + verify_methods + methods writing (extended)

配置: paper_type = "methods"
自动跳过: formulate_hypotheses, figure_planning, write_results, write_introduction, write_discussion
```

4.5 文献综述路径 (Review Paper)

```
跳过 data + analysis + methods + results
保留: literature_search (extended) + introduction + discussion

配置: paper_type = "review"
自动跳过: data_audit, figure_planning, run_analysis, verify_methods, write_methods, write_results
```

---

5. 检查点系统 (6 Human Checkpoints)

```
CP1 — ① select_topic          → "审查研究主题和范围，批准进入文献检索"
CP2 — ④ formulate_hypotheses → "审查假设和实验设计，批准进入数据分析"
CP3 — ⑥ figure_planning       → "审查图表策略和证据逻辑，批准进入分析执行"
CP4 — ⑮ internal_review       → "审查完整手稿和审稿意见，批准进入修订"
CP5 — ⑰ re_review             → "确认修订充分，批准进入最终化"
CP6 — ⑱ finalize              → "最终质量审核，批准提交"
```

决策三态:

- `approved` → 继续
- `revision_needed` → 暂停，等待修复后 `resume()`
- `rejected` → 中止管线

---

6. 容错与恢复机制

6.1 重试策略

```
retry:
  strategy: exponential_backoff
  base_seconds: 60
  multiplier: 2.0          # 1min → 2min → 4min → 8min → 15min (max)
  max_delay_seconds: 900
  jitter: true             # 随机抖动避免雷鸣群效应
  max_cumulative_retries: 20
```

6.2 熔断器 (Circuit Breaker)

```
circuit_breaker:
  failure_threshold: 5       # 5次失败后熔断
  recovery_timeout: 600s     # 10分钟后尝试恢复
  on_open: skip_stage        # 跳过故障阶段，管线继续
```

6.3 过期检测 (Stale Detection)

```
Artifact hash 变更触发链:
  data_audit 更新 data_inventory.yaml (hash 变化)
    → artifact_ledger.jsonl 记录新 hash
    → _sync_stale() 扫描:
        run_analysis.status == COMPLETED
        但 data_audit artifact hash ≠ 记录值
        → run_analysis → STALE
        → 所有下游 (verify_methods, write_*, assemble, ...) → STALE
    → decide_next_stage() 返回 run_analysis (stale 优先)
```

---

7. 文件系统布局

```
papers/<paper_id>/
├── project_passport.yaml          # 项目护照
├── research_plan/                 # Phase 1
│   ├── research_question.md
│   ├── journal_profile.md
│   ├── formatting_requirements.yaml
│   ├── hypotheses.yaml
│   └── feasibility_decision.md
├── references/                    # 文献
│   ├── library.bib
│   └── citation_evidence.csv
├── data/                          # Phase 2
│   ├── data_audit_report.md
│   ├── data_inventory.yaml
│   └── qc_filter_report.md
├── results/                       # 分析结果
│   ├── figure_plan.json
│   ├── clustering_results.yaml
│   ├── de_results.csv
│   └── figures/
├── manuscript/                    # Phase 3-5
│   ├── {abstract,introduction,methods,results,discussion}.md
│   ├── manuscript.tex / .pdf
│   └── manuscript_revised.tex
├── review/                        # Phase 4
│   ├── review_report.md
│   └── re_review_report.md
├── integrity/                     # 质量
│   ├── integrity_report.json
│   └── integrity_report.md
├── submission/                    # Phase 6
│   ├── manuscript_final.pdf
│   └── cover_letter.pdf
├── workflow_state/                # 状态持久化
│   ├── e2e_state_*.json
```

```
│   └── pending_invocations/*.json
├── artifact_ledger.jsonl           # 制品 hash 日志
├── checkpoint_ledger.jsonl         # 检查点决策日志
└── workflow_report.md              # 最终报告
```

---

8. CLI 控制

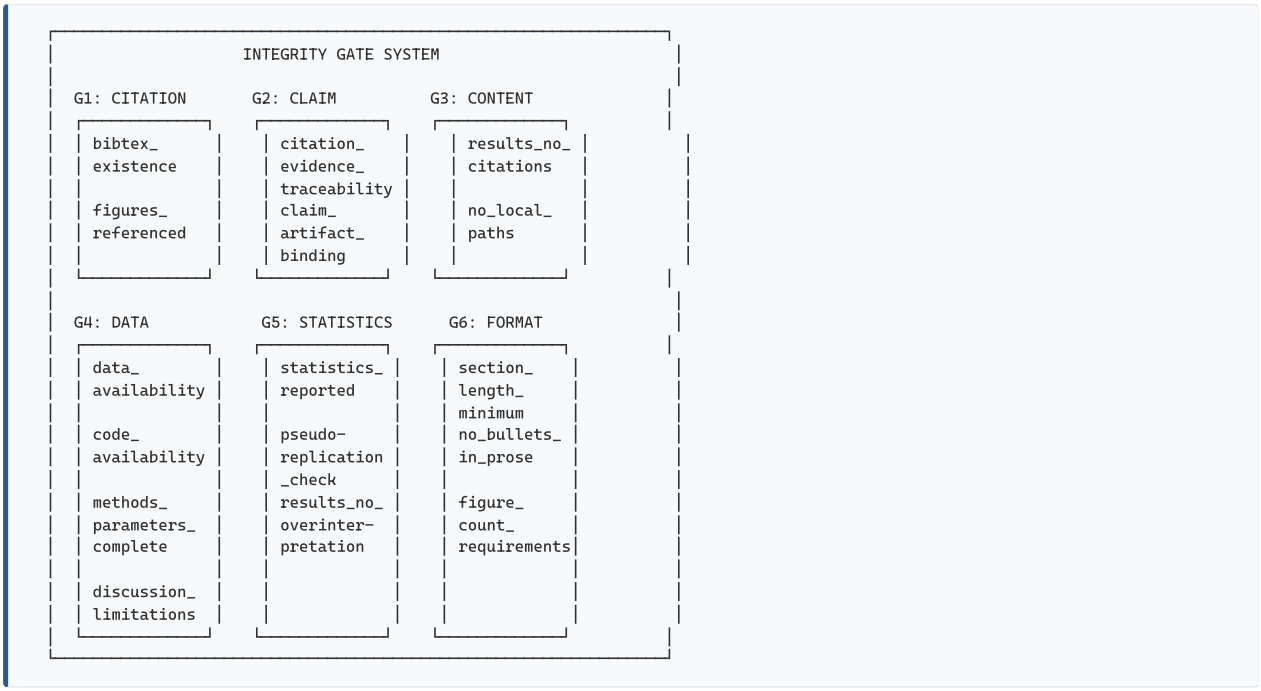
```
# 创建项目
python -m paper_workflow.cli create-project \
  --idea "IgG4-ROD vs MALT lymphoma biomarker discovery" \
  --field "bioinformatics, immunology" \
  --journal "Arthritis Research & Therapy"

# 试运行（不实际执行）
python -m paper_workflow.e2e_workflow --paper-id <id> --dry-run

# 运行特定阶段
python -m paper_workflow.e2e_workflow --paper-id <id> --phases 1,2 --stop-at-checkpoint

# 恢复暂停的工作流
python -m paper_workflow.cli resume --paper <id>

# 诊断 + 导出报告
python -m paper_workflow.cli diagnose --paper <id>
python -m paper_workflow.cli export-report --paper <id>
```

QUALITY_GATES — 16规则质量门体系

> v3.0: 从论文完整性升级为医学证据可信度门控系统。新增5大类别25个医学专用门。

---

1. 质量门概览



---

2. CRITICAL 规则（5条 — 阻塞管线）

G1.1 — bibtex_citation_existence

```
严重级别: CRITICAL ⛔
描述: 手稿中每个引用密钥必须在 references.bib 中存在
检查: 提取所有 \cite{key} → 验证 key ∈ BibTeX entries
阻塞: 是
自动修复: 否 (需人工补充缺失引用)
```

G1.2 — citation_evidence_traceability

```
严重级别: CRITICAL ⛔
描述: 每个事实性声称必须可追溯到至少一条引用证据
检查: 提取声称 → 匹配 citation_evidence.csv → 验证 DOI/PMID
阻塞: 是
自动修复: 否
```

G1.3 — results_no_citations

```
严重级别: CRITICAL ⛔
描述: Results 部分不得包含对外部文献的引用
检查: 扫描 Results 段落 → 检测 \cite{} 或 (Author, Year) 模式
阻塞: 是
自动修复: 否 (需移至 Discussion)
```

G1.4 — claim_artifact_binding

```
严重级别: CRITICAL ⛔
描述: 手稿中每个数值声称必须可追溯到具体的分析制品文件
检查: 提取数值声称 → 匹配 artifact_ledger.jsonl → 验证 hash 一致性
阻塞: 是
自动修复: 否
示例:
  声称: "WGCNA identified 17 co-expression modules"
  追溯: results/wgcna/module_assignment.csv → hash SHA256:abc123 ...
  验证: artifact_ledger.jsonl 中存在该 hash ✓
```

G1.5 — figures_referenced

```
严重级别: CRITICAL ⛔
描述: figures/ 目录中的每个图形文件必须至少在手稿中被引用一次
检查: list figures/ → grep manuscript for each figure filename
阻塞: 是
自动修复: 否
```

---

3. HIGH 规则（8条 — 必须在提交前解决）

G2.1 — data_availability_statement

```
严重级别: HIGH ⚠️
描述: 手稿必须包含完整的数据可用性声明及存储库 accession 号
检查: 搜索 "Data Availability" 段落 → 验证 accession 格式
阻塞: 是 (但支持自动修复)
自动修复: 是 (基于 data_inventory.yaml 生成模板)
```

G2.2 — code_availability_statement

```
严重级别: HIGH ⚠️
描述: 手稿必须包含代码可用性声明及仓库 DOI/URL
检查: 搜索 "Code Availability" 段落 → 验证 DOI/URL 格式
阻塞: 是
自动修复: 是
```

G2.3 — no_local_paths

```
严重级别: HIGH ⚠️
描述: 手稿不得包含本地文件路径 (C:/Users/..., /home/...)
检查: regex 扫描 → 标记路径模式
阻塞: 是
自动修复: 是 (替换为相对路径或通用描述)
```

G2.4 — methods_parameters_complete

```
严重级别: HIGH ⚠️
描述: Methods 必须报告所有软件版本、参数值和随机种子
检查: 解析 Methods → 验证必需字段:
  [software_versions, parameters, random_seeds, thresholds]
阻塞: 是
自动修复: 否 (需人机协作提取参数)
```

G2.5 — discussion_limitations

```
严重级别: HIGH ⚠️
描述: Discussion 必须包含至少一个专门的局限性段落
检查: 在 Discussion 中搜索 "limitation" / "limitations" / "caveat" 关键词
阻塞: 是
自动修复: 否 (需结合具体研究内容)
```

G2.6 — results_no_overinterpretation

```
严重级别: HIGH ⚠️
描述: Results 不得包含因果语言（用于相关性发现）或推测性解释
检查: 扫描因果词汇 ("causes", "leads to", "triggers") + 推测信号
阻塞: 是
自动修复: 否
```

G2.7 — statistics_reported

```
严重级别: HIGH ⚠️
描述: 每个统计检验必须报告: 检验统计量, 效应量, 置信区间, 精确p值
检查: 解析统计声称 → 验证字段完整性:
  [test_statistic, effect_size, ci, p_value]
阻塞: 是
自动修复: 否
```

G2.8 — pseudoreplication_check

```
严重级别: HIGH ⚠️
描述: 无伪重复: 生物重复与技术重复正确区分
检查: 识别重复结构 → 验证独立性假设
阻塞: 是
自动修复: 否
```

---

4. MEDIUM 规则（3条 — 警告不阻塞）

G3.1 — section_length_minimum

```
严重级别: MEDIUM ℹ️
描述: 各部分满足最低字数要求
检查: word_count(section) vs paper_type minimum
阻塞: 否
自动修复: 否
```

G3.2 — no_bullets_in_prose

```
严重级别: MEDIUM ℹ️
描述: 正文段落不得使用项目符号或编号列表
检查: 扫描正文 → 检测 bullet/list markers
阻塞: 否
自动修复: 是 (自动转换为段落)
```

G3.3 — figure_count_requirements

```
严重级别: MEDIUM ℹ️
描述: 图表数量不超过目标期刊限制
检查: count(figures) + count(tables) vs journal limits
阻塞: 否
自动修复: 否
```

---

5. 门触发矩阵

Stage	bibtex	citation_trace	results_no_cite	claim_bind	fig_ref	data_avail	code_avail	no_paths	params	limits	no_overinterp	stats
① select_topic	—	—	—	—	—	—	—	—	—	—	—	—
④ hypotheses	—	—	—	—	—	—	—	—	—	—	—	—
⑧ verify_methods	—	—	—	✓	—	—	—	—	—	—	—	—
⑨ write_methods	—	—	—	—	—	—	—	✓	✓	—	—	—
⑩ write_results	—	—	✓	—	✓	—	—	—	—	—	✓	✓
⑪ write_intro	✓	✓	—	—	—	—	—	—	—	—	—	—
⑫ write_discussion	✓	✓	—	—	—	—	—	—	—	✓	—	—
⑭ integrity_check	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
⑰ re_review	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
⑱ finalize	—	—	—	—	—	✓	✓	—	—	—	—	—

6. 门执行流程

6.1 IntegrityGateChecker 检查顺序

```
class IntegrityGateChecker:
    def run_all_checks(self, manuscript_sections, bibtex_path,
                       figure_plan=None, journal_target=None):
        report = IntegrityReport()

        # Phase 1: Content-independent checks (fast)
        for section_name, content in manuscript_sections.items():
            self._check_no_local_paths(content, report)      # G2.3
            self._check_no_bullets(content, report)          # G3.2
            self._check_section_length(content, report)      # G3.1

        # Phase 2: Citation checks (medium)
        if bibtex_path:
            self._check_bibtex_existence(manuscript_sections,
                                         bibtex_path, report) # G1.1
            self._check_citation_traceability(
                manuscript_sections, bibtex_path, report)     # G1.2

        # Phase 3: Results-specific checks (slowest)
        if 'results' in manuscript_sections:
            self._check_results_no_citations(
                manuscript_sections['results'], report)       # G1.3
            self._check_claim_artifact_binding(
                manuscript_sections['results'],
                self.artifact_ledger, report)                 # G1.4
            self._check_statistics_reported(
                manuscript_sections['results'], report)       # G2.7

        # Phase 4: Structure checks
        self._check_figures_referenced( ... )                 # G1.5
        self._check_data_availability( ... )                  # G2.1
        self._check_code_availability( ... )                  # G2.2
        self._check_methods_parameters( ... )                 # G2.4
        self._check_discussion_limitations( ... )             # G2.5
        self._check_no_overinterpretation( ... )              # G2.6
        self._check_pseudoreplication( ... )                  # G2.8
        self._check_figure_count( ... )                       # G3.3

        # Final: determine blocking status
        report.blocks_pipeline = report.critical_failures > 0
        return report
```

6.2 门结果数据结构

```
@dataclass
class IntegrityReport:
    report_id: str
    passed: bool                    # all CRITICAL passed?
    critical_failures: int
    high_failures: int
    medium_failures: int
    blocks_pipeline: bool           # True if critical_failures > 0
    results: list[GateResult]       # detailed per-rule results

@dataclass
class GateResult:
    rule: str                       # e.g. "bibtex_citation_existence"
    severity: str                   # CRITICAL | HIGH | MEDIUM
    passed: bool
    message: str                    # Human-readable explanation
    violations: list[dict]          # Specific violations with location + fix
```

7. 默认门规则（按阶段自动推导）

当 stage 未显式定义 gate_rules 时，系统按 phase 自动应用默认规则:

```
def _get_default_gate_rules(stage_name, phase):
    if phase ≤ 2:    # Research & Data
        return [{"rule": "all_outputs_exist", "severity": "critical"}]
    elif phase == 3:   # Writing
        return [
            {"rule": "no_local_paths", "severity": "high"},
```

```
            {"rule": "section_length_minimum", "severity": "medium"},
            {"rule": "no_bullets_in_prose", "severity": "medium"},
        ]
    elif phase == 4:   # Assembly & Review
        return [
            {"rule": "bibtex_citation_existence", "severity": "critical"},
            {"rule": "results_no_citations", "severity": "critical"},
            {"rule": "figures_referenced", "severity": "critical"},
        ]
    else:              # Phase 5-6: Revision & Finalize
        return [
            {"rule": "data_availability_statement", "severity": "high"},
            {"rule": "code_availability_statement", "severity": "high"},
            {"rule": "figure_count_requirements", "severity": "medium"},
        ]
```

---

8. 门与管线的交互

```
Stage 执行完成
  │
  ▼
PaperWorkflow._execute_stage(stage)
  ├── engine.run_stage(stage)           # 执行阶段
  ├── engine.verify_stage(stage)        # 运行 gate checks
  │     └── IntegrityGateChecker.run_all_checks()
  │           │
  │           ├── all CRITICAL pass? → StageStatus.COMPLETED
  │           ├── any CRITICAL fail? → StageStatus.FAILED
  │           │     └── PipelineState.GATE_FAILURE
  │           └── MEDIUM fails? → 仅警告, 不阻塞
  │
  ├── passport.record_artifact( ... )     # 记录制品
  └── engine.record_and_sync()            # 同步状态 + 检测过期
```

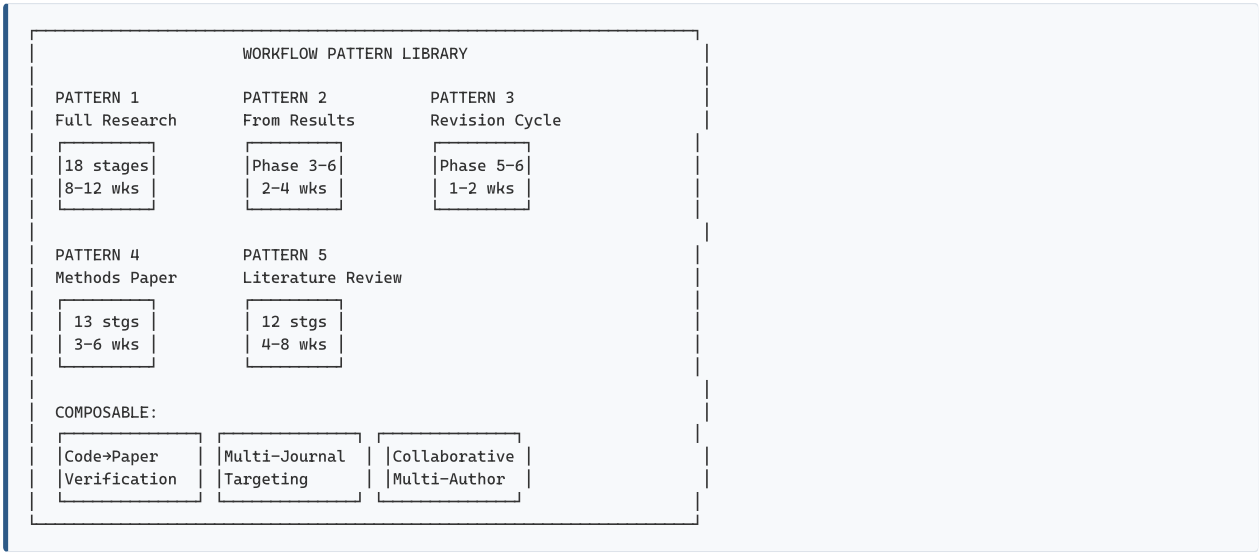
---

9. 自动修复能力

规则	可自动修复	修复方式
data_availability_statement	✅	从 data_inventory.yaml 生成模板
code_availability_statement	✅	从 git remote + 配置生成
no_local_paths	✅	regex 替换本地路径为通用描述
no_bullets_in_prose	✅	将列表转换为段落
bibtex_citation_existence	❌	需人工查找缺失引用
citation_evidence_traceability	❌	需人工验证声称-证据对应
claim_artifact_binding	❌	需人工追溯数值来源
methods_parameters_complete	❌	需人机协作提取参数
discussion_limitations	❌	需基于具体研究撰写
results_no_overinterpretation	❌	需人工判断语义边界

WORKFLOW_PATTERNS — 5种工作流模式

1. 模式总览



2. Pattern 1: Full Research (完整研究路径)

2.1 阶段序列

```
Phase 1: Research & Planning (4 stages)
  ① select_topic [CP]
  ② target_journal ──────┐
  ③ literature_search ──┼──→ ④ formulate_hypotheses [CP]

Phase 2: Data & Methods (4 stages)
  ⑤ data_audit ──┬──→ ⑥ figure_planning [CP]
                 └──→ ⑦ run_analysis → ⑧ verify_methods

Phase 3: Writing (4 stages)
  ⑧ → ⑨ write_methods [CP]
  ⑧+⑥ → ⑩ write_results [CP]
  ③+④ → ⑪ write_introduction [CP]
  ⑩+③ → ⑫ write_discussion [CP]

Phase 4: Assembly & Review (3 stages)
  ⑨+⑩+⑪+⑫ → ⑬ assemble_manuscript [CP]
  ⑬ → ⑭ integrity_check
  ⑭ → ⑮ internal_review [CP]

Phase 5: Revision (2 stages)
  ⑮ → ⑯ apply_revision ↻
       ⑰ re_review [CP] ─┘ (max 5 loops)

Phase 6: Finalize (1 stage)
  ⑰ → ⑱ finalize [CP]
```

2.2 适用范围

条件	说明
数据类型	原始数据在手，需要全流程分析
论文状态	从零开始
团队配置	需要全部 12 个 Agent
时间预算	8-12 周

2.3 调用方式

```
from paper_workflow.e2e_workflow import run_e2e_workflow

wf = run_e2e_workflow(
    paper_id="paper_igg4malt_20260620",
    phases=[1, 2, 3, 4, 5],
    stop_at_checkpoint=True,
    export_report=True
)
```

---

3. Pattern 2: From Results (快速写作路径)

3.1 阶段序列

```
已有: 分析代码 + 结果文件
跳过: Phase 1 策略 + Phase 2 分析执行

Phase 2 (简略):
  ⑤ data_audit (简略 — 仅验证现有制品)
  ⑥ figure_planning [CP]

Phase 3-6: 同 Pattern 1
```

3.2 触发条件

- 分析代码已在项目中
- `results/` 目录已有完整输出
- `artifact_ledger.jsonl` 已有制品记录
- 只需论文写作和审查

3.3 调用方式

```
wf = E2EWorkflow(paper_id="paper_igg4malt_20260620", auto_load=True)
# 自动检测到已有 artifacts，跳过 Phase 1-2
wf.run(phases=[3, 4, 5], stop_at_checkpoint=True)
```

3.4 优化

- `data_audit` 变为纯验证模式（不重新生成）
- `run_analysis` 被替换为 `verify_existing_results`
- 时间从 8-12 周缩短到 2-4 周

---

4. Pattern 3: Revision Cycle (修订循环)

4.1 循环逻辑

```
Reviewer Comments Received
        │
        ▼
  ┌─────────────────────┐
  │ ⑱ apply_revision    │ ← 根据审稿意见修改手稿
  └─────────────────────┘
           │
           ▼
  ┌─────────────────────┐
  │ ⑰ re_review [CP]    │ ← 多视角重审
  └─────────────────────┘
           │
      ┌────┴────┐
      │         │
   PASS        FAIL (retry ≤ 5)
     │           │
     ▼           └──→ 回到 ⑱
  ┌──────────┐
  │ ⑲ finalize│
  └──────────┘
```

4.2 修订智能路由

```
# revision_routing skill 分析审稿意见类型:
revision_type = classify_reviewer_comments(comments)

if revision_type == "minor":
    # 仅修改措辞/格式 → 1次迭代
    wf.run(phases=[5], max_iterations=1)
elif revision_type == "major":
    # 需要补充分析 → 回到 Phase 2 部分阶段
    wf.run(phases=[2, 3, 4, 5])
elif revision_type == "reject_with_resubmit":
    # 重大修改 → 近乎完整重跑
    wf.run(phases=[1, 2, 3, 4, 5])
```

---

5. Pattern 4: Methods Paper (方法学论文)

5.1 核心差异

```
标准 IMRAD → 方法学论文:
  Introduction   → Introduction (含现有方法综述)
  Methods        → Methods (扩展 — 算法详述 + 实现细节)
  Results        → Results (替换为 Validation + Benchmark)
  Discussion     → Discussion (含与其他方法比较 + 适用场景)
```

5.2 自动跳过的阶段

```
# config/default_config.yaml → paper_types.methods.skipped_stages
skipped_stages:
  - formulate_hypotheses    # 方法学无假设检验
  - figure_planning         # 图表更灵活
  - write_results           # 替换为 validation
  - write_introduction      # 结构不同
  - write_discussion        # 替换为 comparison
```

5.3 新增阶段

```
methods_paper 特有:
  benchmark_against_baselines  → 性能对比表
  ablation_study               → 消融实验
  computational_efficiency     → 计算效率报告
  installation_guide           → 安装指南
  api_documentation            → API 文档
```

---

6. Pattern 5: Literature Review (文献综述)

6.1 核心差异

```
跳过: data_audit, figure_planning, run_analysis, verify_methods,
      write_methods, write_results

扩展:
  literature_search → 时间延长至 3600s, 深度搜索多源
  write_introduction → 替换为 thematic_sections (主题分区写作)
  write_discussion → 替换为 synthesis + future_directions
```

6.2 检索策略

```
literature_search (extended for review):
  sources:
    - PubMed (MeSH terms)
    - Semantic Scholar (citation graph)
    - Scopus (author tracking)
    - arXiv (preprints)
    - bioRxiv (preprints)
  dedup_method: "DOI-based + title fuzzy match"
  quality_filter:
    - min_citations: 5
    - peer_reviewed_only: true
```

```
    - exclude_predatory: true
  synthesis_method: "thematic clustering + gap analysis"
```

---

7. 可组合模式

7.1 Code→Paper Verification (代码-论文交叉验证)

```
目的: 确保代码输出与论文声称 100% 一致

流程:
  1. 提取论文中所有数值声称
  2. 运行分析代码
  3. 逐项比对:
     ┌─ 统计量 (β, OR, HR, p-value)
     ├─ 样本量 (n= ... )
     ├─ 阈值 (FDR<0.05, |log2FC⊳1)
     └─ 模块/聚类数
  4. 生成差异报告
  5. 自动修复可修复的不一致
```

7.2 Multi-Journal Targeting (多期刊适配)

```
目的: 同一研究快速适配不同期刊格式

流程:
  1. 定义核心内容 (研究问题+结果+方法)
  2. 对每个目标期刊:
     ├─ 调整 abstract (字数限制)
     ├─ 调整 图表数量
     ├─ 重新格式化引用
     └─ 调整 Discussion 侧重点
  3. 并行生成多版本手稿
```

7.3 Collaborative Multi-Author (多作者协作)

```
目的: 支持分布式作者团队

流程:
  1. team_orchestrator 分配写作任务:
     ├─ Author A → Introduction
     ├─ Author B → Methods
     ├─ Author C → Results (含图表)
     └─ Author D → Discussion
  2. 并行写作 (各作者在自己的工作树中)
  3. assemble_manuscript 合并
  4. integrity_check 验证一致性
  5. internal_review 交叉审查
```

---

8. 模式选择决策树

```
START
  │
  ├─ 有新数据需要分析?
  │   ├─ YES → Pattern 1: Full Research
  │   └─ NO  → 是否有论文初稿?
  │             ├─ YES → 是否需要修订?
  │             │         ├─ YES → Pattern 3: Revision Cycle
  │             │         └─ NO  → Pattern 2: From Results
  │             └─ NO  → 论文类型?
  │                       ├─ 方法学/工具 → Pattern 4: Methods
  │                       ├─ 文献综述    → Pattern 5: Review
  │                       └─ 原始研究    → Pattern 1: Full Research
  │
```

---

9. 模式性能特征

Pattern	阶段数	Agent 数	并行度	典型耗时	Token 用量
Full Research	18	12	高 (4-5 并行)	8-12 周	~500K
From Results	12	7	高 (3-4 并行)	2-4 周	~200K
Revision Cycle	3	4	中 (2 并行)	1-2 周	~100K
Methods Paper	13	8	中 (2-3 并行)	3-6 周	~300K
Literature Review	12	5	低 (串行为主)	4-8 周	~400K

---

10. 模式组合示例

场景: 先做 Full Research，收到审稿意见后进入 Revision Cycle

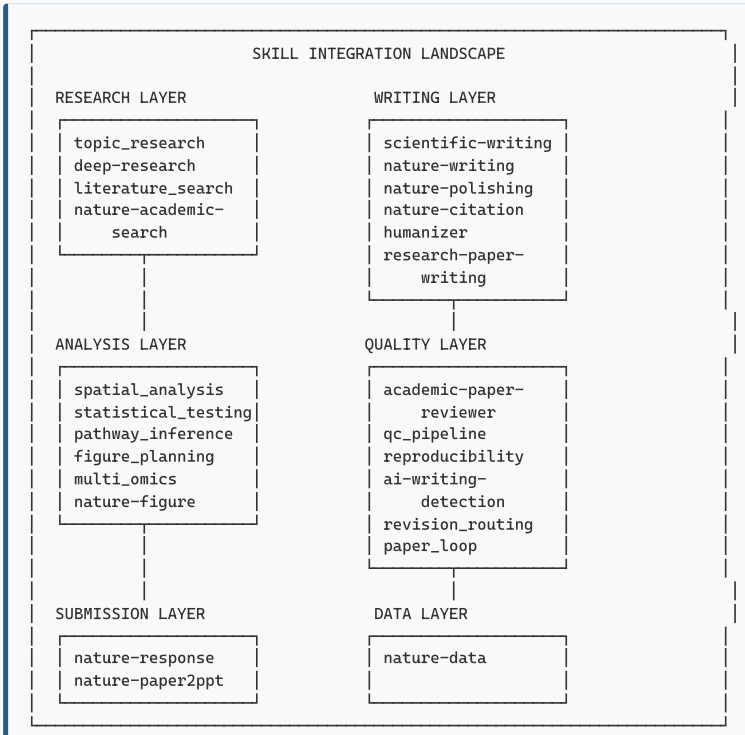
```
# Round 1: Full Research
wf1 = run_e2e_workflow("paper_igg4malt", phases=[1,2,3,4,5])
# 提交 → 收到审稿意见

# Round 2: Revision Cycle
wf2 = E2EWorkflow("paper_igg4malt", auto_load=True)
wf2.run(phases=[5])  # 仅修订+重审+定稿

# 如果是重大修改:
wf2.run(phases=[2, 3, 4, 5])  # 从分析验证重新开始
```

场景: Code→Paper Verification + Revision Cycle

```
# 先交叉验证
verify_code_paper_consistency(paper_id="paper_igg4malt")

# 如有不一致，触发修订
if inconsistencies_found:
    wf = E2EWorkflow("paper_igg4malt", auto_load=True)
    wf.run(phases=[5])
```

INTEGRATION_MAP — 技能集成与工具矩阵

1. 技能全景映射



2. 内部技能 (12 Framework Skills)

2.1 技能定义文件

技能	定义文件	Agent	Phase
`topic_research`	`.claude/skills/topic_research.md`	research_strategist	1
`literature_search`	`.claude/skills/literature_search.md`	literature_reviewer	1
`qc_pipeline`	`.claude/skills/qc_pipeline.md`	integrity_checker	2,4,5
`figure_planning`	`.claude/skills/figure_planning.md`	figure_planner	2
`spatial_analysis`	`.claude/skills/spatial_analysis.md`	analysis_executor	2
`statistical_testing`	`.claude/skills/statistical_testing.md`	statistician	2
`pathway_inference`	`.claude/skills/pathway_inference.md`	analysis_executor	2
`multi_omics`	`.claude/skills/multi_omics.md`	multi_omics_integrator	2
`paper_writing`	`.claude/skills/paper_writing.md`	report_writer	3
`paper_loop`	`.claude/skills/paper_loop.md`	team_orchestrator	4
`revision_routing`	`.claude/skills/revision_routing.md`	report_writer	5
`reproducibility`	`.claude/skills/reproducibility.md`	pipeline_engineer	2,6

2.2 技能调度触发器（来自 `default_config.yaml §6`）

```
# 关键触发器映射
deep-research:
  triggers: [literature search, comprehensive review, state of the art,
```

```
            deep调研, 系统性检索, 文献综述]

scientific-writing:
  triggers: [write Methods, write Results, write Introduction,
             write Discussion, IMRAD, 医学论文, 科研写作段落]

nature-polishing:
  triggers: [polish manuscript, improve language, 润色, 改写, proofreading]

nature-citation:
  triggers: [add citations, insert references, Nature系列引用,
             CNS子刊, 补引用, 配文献]

humanizer:
  triggers: [remove AI traces, 去AI味, 不像AI写的, 更自然]

academic-paper-reviewer:
  triggers: [review manuscript, peer review, 审稿, critique paper]
```

---

3. 外部技能集成 (28 External Skills)

3.1 按阶段分组

阶段	外部技能	用途
Phase 1	`deep-research`	多源文献调研 + 引用追踪
	`nature-academic-search`	PubMed/CrossRef/arXiv 检索
	`summarize`	论文快速理解
	`paper-glance`	论文深度分析 + 审稿意见
Phase 2	`nature-figure`	发表级图表 (≥300 DPI)
	`nature-data`	FAIR 数据管理
Phase 3	`scientific-writing`	IMRAD 结构段落写作
	`nature-writing`	Nature 风格手稿构建
	`nature-citation`	CNS/子刊引用集成
	`research-paper-writing`	ML/CV/NLP 风格论文
	`academic-paper`	12-Agent 学术写作管线
Phase 4	`academic-paper-polish`	学术润色
	`nature-polishing`	Nature 风格润色 + LaTeX 排版
	`humanizer`	AI 痕迹移除
	`academic-paper-reviewer`	多视角内审 (EIC+3Reviewers+Devil)
	`ai-writing-detection`	AI 写作检测
	`remove-ai-flavor`	中文 AI 味道去除
Phase 5	`nature-response`	审稿回复信
	`academic-paper-reviewer`	重审 (re-review mode)
Phase 6	`nature-paper2ppt`	论文转 PPT
	`nature-data`	数据可用性声明

3.2 完整外部技能列表

```
academic-paper          academic-paper-polish     academic-paper-reviewer
academic-pipeline       ai-writing-detection      darwin-skill
deep-research           find-skills               gws-docs
humanizer               nature-academic-search    nature-citation
nature-data             nature-figure             nature-paper2ppt
nature-polishing        nature-reader             nature-response
nature-writing          paper-glance              remove-ai-flavor
research-paper-writing  scientific-writing        skill-creator
summarize               tavily-research
```

4. MCP 工具集成

4.1 MCP Server 矩阵

MCP Server	工具	用途	频率
consensus	`search`	2亿+学术论文搜索，引用计数+期刊质量	Phase 1
context7	`resolve-library-id`，`query-docs`	R/Python 库最新文档查询	Phase 2
exa	`web_search_exa`，`web_fetch_exa`	网页搜索+内容提取	Phase 1,5
grok-search	`web_search`，`web_fetch`，`web_map`	通用搜索+网站地图	All phases
MiniMax	`web_search`，`understand_image`	通用搜索+图像理解	Phase 2,4
fast-context	`fast_context_search`	AI 驱动代码语义搜索	Phase 2
pubmed	`search_articles`	PubMed 生物医学文献	Phase 1

4.2 按任务类型的工具选择

任务	首选工具	备选工具
生物医学文献搜索	`mcp__pubmed__search_articles`	`mcp__consensus__search`
跨学科学术搜索	`mcp__consensus__search`	`mcp__exa__web_search_exa`
R/Bioconductor 文档	`mcp__context7__query-docs`	Web search
期刊投稿要求	`mcp__grok-search__web_search`	`mcp__exa__web_search_exa`
代码语义搜索	`mcp__fast-context__fast_context_search`	Grep
论文全文获取	`mcp__exa__web_fetch_exa`	`mcp__grok-search__web_fetch`
图形内容分析	`mcp__MiniMax__understand_image`	—
网站结构分析	`mcp__grok-search__web_map`	—

5. 阶段 → 技能 → 工具 路由表

5.1 Phase 1: Research & Planning

```
Stage ① select_topic
  ├── Skill: topic_research
  │     └── MCP: grok-search (最新研究趋势)
  └── Skill: deep-research
        └── MCP: consensus + pubmed (全方位检索)

Stage ② target_journal
  ├── Skill: topic_research
  │     └── MCP: grok-search (期刊 Guide for Authors)
  └── Skill: nature-academic-search
        └── MCP: pubmed (期刊范围+发表记录)

Stage ③ literature_search
  ├── Skill: deep-research
  │     └── MCP: consensus + pubmed (系统检索)
  ├── Skill: nature-academic-search
  │     └── MCP: pubmed (MeSH 检索策略)
  └── Skill: nature-citation
        └── MCP: pubmed (引用验证)

Stage ④ formulate_hypotheses
  ├── Skill: topic_research
  └── Skill: scientific-writing
```

5.2 Phase 2: Data & Methods

```
Stage ⑤ data_audit
  ├── Skill: nature-data (FAIR 检查)
  └── Skill: qc_pipeline (QC 指标验证)
```

```
Stage ⑥ figure_planning
  ├── Skill: figure_planning (图表策略)
  └── Skill: nature-figure (发表标准)

Stage ⑦ run_analysis
  ├── Skill: spatial_analysis
  ├── Skill: statistical_testing
  ├── Skill: pathway_inference
  │     └── MCP: context7 (R包文档参考)
  └── MCP: fast-context (代码定位+调试)

Stage ⑧ verify_methods
  ├── Skill: qc_pipeline (输出验证)
  ├── Skill: reproducibility (环境快照)
  └── Skill: statistical_testing (统计审核)
```

5.3 Phase 3: Writing

```
Stage ⑨–⑫ Write IMRAD
  ├── Skill: scientific-writing (段落草稿)
  ├── Skill: nature-writing (Nature 风格)
  ├── Skill: nature-citation (引用集成)
  │     └── MCP: pubmed (DOI/PMID 验证)
  └── Skill: research-paper-writing (claim-support 对齐)
```

5.4 Phase 4: Assembly & Review

```
Stage ⑬ assemble_manuscript
  ├── Skill: paper_writing
  ├── Skill: nature-figure (图形嵌入)
  └── Skill: nature-citation (引用编号)

Stage ⑭ integrity_check
  └── Skill: qc_pipeline (16 gate rules)

Stage ⑮ internal_review
  ├── Skill: academic-paper-reviewer (5 reviewer personas)
  └── Skill: humanizer (AI traces check)
```

5.5 Phase 5-6: Revision & Finalize

```
Stage ⑯ apply_revision
  ├── Skill: paper_writing
  ├── Skill: nature-polishing
  ├── Skill: humanizer
  └── Skill: nature-response

Stage ⑰ re_review
  └── Skill: academic-paper-reviewer (re-review mode)

Stage ⑱ finalize
  ├── Skill: qc_pipeline (最终完整性)
  ├── Skill: nature-data (数据声明)
  ├── Skill: nature-polishing (最终润色)
  └── Skill: nature-paper2ppt (可选)
```

---

6. Agent → 技能 → MCP 分配矩阵

Agent	Internal Skills	External Skills	MCP Tools
research_strategist	topic_research	deep-research, nature-academic-search	consensus, grok-search, pubmed
literature_reviewer	literature_search	deep-research, nature-academic-search, nature-citation	consensus, pubmed
data_auditor	qc_pipeline	nature-data	context7
figure_planner	figure_planning	nature-figure	—
analysis_executor	spatial_analysis, pathway_inference, qc_pipeline	—	fast-context, context7
pipeline_engineer	qc_pipeline, reproducibility	—	context7
statistician	statistical_testing	—	—
report_writer	paper_writing, revision_routing	scientific-writing, nature-writing, nature-polishing, nature-citation, nature-response, humanizer, research-paper-writing	pubmed
integrity_checker	qc_pipeline	academic-paper-reviewer, nature-data, ai-writing-detection	—
multi_omics_integrator	multi_omics, spatial_analysis, pathway_inference	—	context7
code_librarian	qc_pipeline	—	fast-context
team_orchestrator	paper_loop	deep-research, academic-pipeline	grok-search

7. Skill 注册表（`.claude/SKILL_REGISTRY.md`）

7.1 触发词索引

基于 `config/default_config.yaml §6 (skills_dispatcher)`，每个 skill 有完整的中英文触发词列表:

```
# 示例: scientific-writing 触发词
scientific-writing:
  triggers:
    english:
      - "write Methods"
      - "write Results"
      - "write Introduction"
      - "write Discussion"
      - "draft section"
      - "IMRAD"
      - "scientific manuscript"
      - "draft manuscript"
      - "reporting guidelines"
      - "CONSORT"
      - "STROBE"
    chinese:
      - "医学论文"
      - "科研写作段落"
      - "写论文段落"
```

7.2 自动触发机制

```
用户输入 → 技能触发词匹配 → SkillsDispatcher
  ├── 精确匹配 → 直接调度
  ├── 模糊匹配 → 列出候选, 请求确认
  └── 无匹配 → 回退到 team_orchestrator (通用 Agent)
```

8. 扩展集成

8.1 添加自定义技能

```
# config/custom_skills.yaml
my_custom_analysis:
  triggers:
    - "custom analysis"
    - "my specific method"
  phase: "analysis"
```

```
  agent: "analysis_executor"
  description: "My specialized analysis pipeline"
```

8.2 添加新 MCP Server

```
// .claude/mcp.json
{
  "mcpServers": {
    "my-new-server": {
      "command": "my-mcp-server",
      "args": ["--config", "path/to/config"]
    }
  }
}
```

CASE_STUDIES — 3个真实项目案例

Case 1: IgG4&MALT — 生物标志物发现 (WGCNA + ML)

项目背景

属性	值
研究问题	IgG4-ROD vs MALT Lymphoma 鉴别诊断生物标志物
数据类型	转录组 (microarray/RNA-seq)
分析方法	DE分析 → CIBERSORT免疫浸润 → WGCNA → ML
目标期刊	Arthritis Research & Therapy
当前状态	WGCNA完成 (17 modules, 132 hub-DEG)，待ML + 论文写作

管线映射

```
已完成的阶段：

Phase 1: Research & Planning
  ① select_topic ✓ — "IgG4-ROD vs MALT differential biomarkers"
  ③ literature_search ✓ — IgG4-RD, MALT lymphoma, WGCNA biomarker studies

Phase 2: Data & Methods
  ⑤ data_audit ✓ — GEO dataset validated
  ⑦ run_analysis (partial):
    ├── DE analysis ✓ — 842 DEGs identified
    ├── CIBERSORT ✓ — 6 immune cell types significant
    └── WGCNA ✓ — power=20, 17 modules, 8 disease-immune, 132 hub-DEG

待完成的阶段：

Phase 1: ② target_journal, ④ formulate_hypotheses
Phase 2: ⑥ figure_planning, ⑦ run_analysis (ML部分), ⑧ verify_methods
Phase 3-6: 全文写作 + 审查 + 修订 + 定稿
```

工作流执行计划

```
# 当前项目快速写作路径 (From Results pattern)
wf = E2EWorkflow(paper_id="paper_igg4malt_20260620")

# Phase 1: 补完策略
wf.run(phases=[1])  # target_journal + hypotheses (已有 topic + literature)

# Phase 2: 补完分析
wf.run(phases=[2])  # figure_planning + ML analysis + verify

# Phase 3-6: 写作+审查+定稿
wf.run(phases=[3, 4, 5], stop_at_checkpoint=True)
```

特定配置覆盖

```
# paper_config.yaml (项目级覆盖)
paper_type: original_research
target_journal: "Arthritis Research & Therapy"

writing_standards:
  sections:
    abstract:
      max_words: 300
    methods:
      subsections:
        - "Data Acquisition and Preprocessing"
        - "Differential Expression Analysis"
        - "Immune Cell Deconvolution (CIBERSORT)"
        - "Weighted Gene Co-expression Network Analysis (WGCNA)"
        - "Machine Learning-Based Feature Selection"
        - "Diagnostic Model Construction and Validation"
```

关键质量门关注点

- **G1.4 (claim_artifact_binding)**: WGCNA power=20, 17 modules, 132 hub-DEG — 必须可追溯到 `results/wgcna/` 下的具体文件
- **G2.4 (methods_parameters_complete)**: WGCNA 参数必须完整 — softPower, minModuleSize, mergeCutHeight, signedKME 等
- **G2.7 (statistics_reported)**: DE 结果必须报告 log2FC + CI + adjusted p-value

---

Case 2: 肝骨轴MR — 孟德尔随机化中介分析

项目背景

属性	值
研究问题	验证"肝骨轴"因果假说: LiverPDFF → BMFF → BMD
数据类型	GWAS summary statistics (公开)
分析方法	两样本MR → 中介MR (四步) → MVMR → 敏感性分析
核心结果	LiverPDFF→BMFF β=+0.0621(P=4e-5), BMFF→BMD β=-0.0883(P=2e-7), 中介 α×β=-0.00548(P=0.001)
当前状态	v17.5 主分析完成，论文待整合

管线映射

```
Phase 2: Data & Methods
  ⑤ data_audit — GWAS catalog + OpenGWAS 数据验证
  ⑦ run_analysis:
    ├── Two-sample MR (IVW, MR-Egger, Weighted Median, MR-PRESSO)
    ├── Bidirectional MR (排除反向因果)
    ├── Two-step Mediation MR (中介效应)
    ├── Multivariable MR (竞争性中介)
    └── Sensitivity: Cochran's Q, Leave-one-out, Steiger, F-statistic
  ⑧ verify_methods:
    └── Gate G2.8 (pseudoreplication_check) — 对 MR 特别重要:
        确保暴露和结局 GWAS 样本不重叠
        确保 IV 选择无 winner's curse

Phase 3: Writing
  ⑨ write_methods — 需遵循 STROBE-MR 报告指南
  ⑩ write_results — 每个 β 必须带 CI + p-value + 敏感性检验结果
  ⑪ write_introduction — 肝骨轴生物学背景 + MR 方法学空白
  ⑫ write_discussion — 中介比例解释 + 竞争性中介讨论
```

特定质量门

```
MR 特定的额外检查:
  □ F-statistic ≥ 10 for all IVs (弱工具变量排除)
  □ MR-Egger intercept p > 0.05 (无显著水平多效性)
  □ MR-PRESSO global test (异常值检测)
  □ Cochran's Q p > 0.05 (无异质性)
  □ Steiger directionality: "TRUE" (因果方向正确)
  □ 中介效应 Sobel test + Bootstrap CI
```

工作流执行

```
wf = E2EWorkflow(paper_id="paper_liver_bone_axis_mr")

# 快速写作路径 (已有完整分析结果)
wf.run(phases=[3, 4, 5], stop_at_checkpoint=True)

# 如果审稿人要求额外敏感性分析:
wf.run(phases=[2, 3, 4, 5])  # 回到 Phase 2 补充分析
```

Case 3: StereoSeq — 空间转录组学平台

项目背景

属性	值
研究问题	健康肾脏空间转录组参考图谱 → 空间解卷积 → 衰老通路机制
数据类型	Stereo-seq bin50 (30,904 spots × 34,363 genes) + scRNA-seq 参考 (GSE185809, 56,728 cells)
分析方法	QC → scRNA参考图谱 → 空间解卷积 → 区域分割 → PCD通路分析
当前状态	全部分析完成，Methods section 修订中

管线映射

```
Phase 1:
  ① select_topic — "Spatial transcriptomic atlas of aging human kidney"
  ③ literature_search — 空间转录组肾脏研究, 解卷积方法比较

Phase 2 (已完成全部分析):
  ⑤ data_audit — bin50 QC: MT<25%/30%, Leiden r=0.6
  ⑦ run_analysis:
    ├── scRNA reference (5 samples, 19 cell types, KIDNEY_COLORS)
    ├── Spatial deconvolution (SpecWeight NNLS + Anatomical Prior + Sinkhorn-Knopp, MAE=0.19%)
    ├── Area domain segmentation (3-area: glomeruli/cortex/medulla)
    ├── PCD pathway analysis (18 pathways × 3 areas × 2 samples, 42/54 significant)
    └── Spatial expression visualization (星云热图 144 PDF + 通路等高线 12图)
  ⑧ verify_methods:
    └── Gate G2.4 — 参数完整性至关重要:
        解卷积方法选择 (SpecWeight>DWLS 的论据)
        区域分割参数 (graph morphological)
        PCD 通路评分方法

Phase 3 (写作中):
  ⑨ write_methods:
    └── 必须精确记录:
        - 空间解卷积: SpecWeight NNLS 参数, Anatomical Prior 构建
        - 区域分割: 图形态学分割方法, boundary detection
        - PCD 分析: 18种细胞死亡通路评分, FDR 校正
```

方法学论文特征

```
paper_type: methods  # 方法学/平台论文

# 重点:
# - 解卷积 Benchmark: SpecWeight vs DWLS vs RCTD vs SPOTlight
# - 计算效率: 30,904 spots 处理时间
# - 可复现性: Docker + renv.lock + 固定 seeds
# - 数据可用性: GEO accession (参考数据) + Zenodo (处理数据)
```

关键交叉验证点

```
论文 Methods vs 代码:
  □ "MT<25%" — mt_filter.py 默认值 25 ✓
  □ "Leiden r=0.6" — leiden_clustering.py 默认值 0.6 ✓
  □ "SpecWeight NNLS" — 实际使用的方法 vs DWLS (曾尝试但效果差)
  □ "MAE=0.19%" — deconvolution 评估指标
  □ "42/54 significant" — PCD 分析 FDR<0.05 统计
  □ "Anoikis δ=0.55-0.67" — 效应量报告
```

Case 4: test_e2e_live — 端到端测试案例

测试概述

位于 `papers/test_e2e_live/`，展示了完整的项目文件结构:

```
papers/test_e2e_live/
├── project_passport.yaml
├── checkpoint_ledger.jsonl
├── artifact_ledger.jsonl
├── research_plan/
│   ├── feasibility_decision.md
│   ├── formatting_requirements.yaml
│   └── journal_profile.md
├── data/
│   ├── data_audit_report.md
│   ├── data_inventory.yaml
│   └── qc_filter_report.md
```

```
├── results/
│   ├── cell_annotation.md
│   └── clustering_results.yaml
├── integrity/
│   ├── integrity_report.json
│   └── integrity_report.md
└── workflow_state/
    ├── e2e_state_20260618_170203.json
    └── pending_invocations/  (5 个 skill 调用等待)
```

测试覆盖

```
# tests/test_all.py 和 tests/test_integration.py
# 测试内容:
# - P0 verification: 核心流程完整性
# - E2E workflow: 5 phase 全流程
# - Integrity gates: 16 rules 执行
# - Passport: artifact 哈希记录+验证
# - Agent dispatch: 12 agent 调度
# - Config loading: YAML 配置解析
```

跨案例对比

维度	IgG4&MALT	肝骨轴MR	StereoSeq
研究类型	生物标志物发现	因果推断	方法学/平台
论文类型	original_research	original_research	methods
数据来源	GEO (转录组)	GWAS (公开)	Stereo-seq + scRNA-seq
核心方法	WGCNA + ML	MR + Mediation	Deconv + Spatial
报告指南	—	STROBE-MR	—
关键Gate	G2.4 (WGCNA参数)	G2.8 (样本独立性)	G2.4 (解卷积参数)
工作流模式	Pattern 2 (From Results)	Pattern 2 (From Results)	Pattern 4 (Methods Paper)
当前阶段	Phase 2→3 过渡	Phase 3 写作	Phase 3 写作
预估完成	4-6 周	2-4 周	2-4 周

QUICK_REFERENCE — 一页速查卡

工作流命令

```
# 创建新论文项目
python -m paper_workflow.cli create-project \
  --idea "your idea" --field "your field" --journal "target journal"

# 试运行（查看计划不执行）
python -m paper_workflow.e2e_workflow --paper-id <id> --dry-run

# 运行指定阶段
python -m paper_workflow.e2e_workflow --paper-id <id> --phases 1,2

# 运行全部5阶段（自动模式）
python -m paper_workflow.e2e_workflow --paper-id <id> --phases 1,2,3,4,5

# 检查点模式（每阶段暂停）
python -m paper_workflow.e2e_workflow --paper-id <id> --stop-at-checkpoint

# 查看状态
python -m paper_workflow.cli status --paper <id>

# 诊断失败
python -m paper_workflow.cli diagnose --paper <id>

# 导出报告
python -m paper_workflow.cli export-report --paper <id>
```

18 Stage Pipeline 速查

#	Stage	Phase	Agent	CP
①	select_topic	1:Research	research_strategist	✓
②	target_journal	1:Research	research_strategist	—
③	literature_search	1:Research	literature_reviewer	—
④	formulate_hypotheses	1:Research	research_strategist	✓
⑤	data_audit	2:Data	data_auditor	—
⑥	figure_planning	2:Data	figure_planner	✓
⑦	run_analysis	2:Data	analysis_executor	—
⑧	verify_methods	2:Data	pipeline_engineer	—
⑨	write_methods	3:Writing	report_writer	✓
⑩	write_results	3:Writing	report_writer	✓
⑪	write_introduction	3:Writing	report_writer	✓
⑫	write_discussion	3:Writing	report_writer	✓
⑬	assemble_manuscript	4:Assembly	report_writer	✓
⑭	integrity_check	4:Assembly	integrity_checker	—
⑮	internal_review	4:Assembly	team_orchestrator	✓
⑯	apply_revision	5:Revision	report_writer	—
⑰	re_review	5:Revision	team_orchestrator	✓
⑱	finalize	6:Finalize	integrity_checker	✓

12 Agents 速查

Agent	负责阶段	核心技能
research_strategist	①②④	topic_research, deep-research
literature_reviewer	③	deep-research, nature-citation
data_auditor	⑤	nature-data, qc_pipeline
figure_planner	⑥	figure_planning, nature-figure
analysis_executor	⑦	spatial_analysis, pathway_inference
pipeline_engineer	⑦⑧	qc_pipeline, reproducibility
statistician	⑧	statistical_testing
report_writer	⑨⑩⑪⑫⑬⑯	scientific-writing, nature-*, humanizer
integrity_checker	⑧⑭⑱	qc_pipeline, academic-paper-reviewer
multi_omics_integrator	⑦⑧	multi_omics, spatial_analysis
code_librarian	⑦⑧⑱	qc_pipeline
team_orchestrator	⑮⑰	deep-research (协调)

16 Quality Gates 速查

#	Gate	Severity	Auto-Fix
1	bibtex_citation_existence	CRITICAL	❌
2	citation_evidence_traceability	CRITICAL	❌
3	results_no_citations	CRITICAL	❌
4	claim_artifact_binding	CRITICAL	❌
5	figures_referenced	CRITICAL	❌
6	data_availability_statement	HIGH	✅
7	code_availability_statement	HIGH	✅
8	no_local_paths	HIGH	✅
9	methods_parameters_complete	HIGH	❌
10	discussion_limitations	HIGH	❌
11	results_no_overinterpretation	HIGH	❌
12	statistics_reported	HIGH	❌
13	pseudoreplication_check	HIGH	❌
14	section_length_minimum	MEDIUM	❌
15	no_bullets_in_prose	MEDIUM	✅
16	figure_count_requirements	MEDIUM	❌

5 Workflow Patterns

Pattern	阶段	耗时	适用场景
Full Research	1-6	8-12wk	新项目从零开始
From Results	3-6	2-4wk	分析完成, 只需写作
Revision Cycle	5-6	1-2wk	审稿修回
Methods Paper	1-6 (adapted)	3-6wk	方法学/工具论文
Literature Review	1-4 (adapted)	4-8wk	文献综述

5 E2E Phases

Phase	名称	阶段数	检查点
1	Topic Research	4	✓
2	Data Analysis	4+3code	✓
3	Paper Writing	4	✓
4	Polish & Review	3+6ext	✓
5	Submission	3	✓

关键 MCP 工具速查

任务	工具调用
生物医学文献	`mcp__pubmed__search_articles`
跨学科学术	`mcp__consensus__search`
R包文档	`mcp__context7__resolve-library-id` → `mcp__context7__query-docs`
代码语义搜索	`mcp__fast-context__fast_context_search`
期刊政策搜索	`mcp__grok-search__web_search`
论文全文	`mcp__exa__web_fetch_exa`

项目记忆参照

项目	记忆文件	关键信息
IgG4&MALT	`igg4malt_wgcna_20260513`	WGCNA 17模块, 132 hub-DEG
肝骨轴MR	`肝骨轴MR/`	MVMR v17.5, β/CI完整
StereoSeq	`stereoseq_*` (多个)	空间解卷积, PCD分析
肾脏多组学	`kidney_multiomics_survey_2026`	调研阶段

常见操作

从已有结果快速写论文

```
from paper_workflow.e2e_workflow import E2EWorkflow
wf = E2EWorkflow(paper_id="paper_my_project", auto_load=True)
wf.run(phases=[3, 4, 5], stop_at_checkpoint=True)
```

只写 Methods 段落

```
wf = E2EWorkflow(paper_id="paper_my_project", auto_load=True)
wf.run(phases=[3])  # Phase 3 第一个 stage 就是 write_methods
```

修订审稿意见

```
wf = E2EWorkflow(paper_id="paper_my_project", auto_load=True)
wf.run(phases=[5])  # apply_revision → re_review ↻
```

重新运行分析后更新论文

```
# 1. 重新分析
wf.run(phases=[2])
# 2. 门检测到 artifact hash 变化 → 所有下游 STALE
```

```
# 3. 自动重新写作 (stale 优先)
wf.run(phases=[3, 4, 5])
```